Introduction to Attention and Transformer Architecture

Balaji Srinivasan, Ganapathy Krishnamurthi

Wadhwani School of AI , IIT Madras

Outline of this Module

1. Word Embeddings
2. The Attention Module
3. Transformer Architecture
4. Transformers for Vision
5. Advanced models with Transformers

Word Embeddings

The Building Blocks of Language Models

The Problem: Computers Don't Understand Words

Computers operate on numbers, not text. They don't inherently know that "happy" is the opposite of "sad" or that a "cat" is a type of "pet".

The Problem: Computers Don't Understand Words

Computers operate on numbers, not text. They don't inherently know that "happy" is the opposite of "sad" or that a "cat" is a type of "pet".

How can we represent the *meaning* of words in a way a computer can process?

The Solution: Word Embeddings

A **word embedding** is a way of representing a word as a list of numbers, called a **vector**. Each word in our vocabulary gets its own unique vector.

The Solution: Word Embeddings

A **word embedding** is a way of representing a word as a list of numbers, called a **vector**. Each word in our vocabulary gets its own unique vector.

- `King` → `[0.12, -0.45, 0.88, ..., -0.15]`
- `Queen` → `[0.82, -0.41, 0.85, ..., 0.55]`
- `Apple` → `[-0.53, 0.91, -0.04, ..., 0.73]`

The Solution: Word Embeddings

A **word embedding** is a way of representing a word as a list of numbers, called a **vector**. Each word in our vocabulary gets its own unique vector.

- `King` → `[0.12, -0.45, 0.88, ..., -0.15]`
- `Queen` → `[0.82, -0.41, 0.85, ..., 0.55]`
- `Apple` → `[-0.53, 0.91, -0.04, ..., 0.73]`

Think of these vectors as coordinates for each word in a high-dimensional "meaning space."

What Do These Numbers Capture?

The numbers in the vector aren't random; they capture the **semantic meaning** and **context** of a word.

What Do These Numbers Capture?

The numbers in the vector aren't random; they capture the **semantic meaning** and **context** of a word.

This means that words with similar meanings will have similar vectors. The distance between two vectors tells us how similar the corresponding words are.

What Do These Numbers Capture?

The numbers in the vector aren't random; they capture the **semantic meaning** and **context** of a word.

This means that words with similar meanings will have similar vectors. The distance between two vectors tells us how similar the corresponding words are.

Key Idea

The vector for "cat" will be mathematically closer to the vector for "dog" than it is to the vector for "car".

Visualizing Embeddings: A Map of Words

If we simplify the high-dimensional "meaning space" to 2D, we can see how words cluster together based on their meaning.

Visualizing Embeddings: A Map of Words

If we simplify the high-dimensional "meaning space" to 2D, we can see how words cluster together based on their meaning.

Figure: Words with similar meanings are grouped together.

Embeddings Capture Relationships

The directions between vectors can capture relationships. The most famous example is the male-female to royalty relationship.

Embeddings Capture Relationships

The directions between vectors can capture relationships. The most famous example is the male-female to royalty relationship.

The Male-Female royalty example

The vector relationship between 'King' and 'Queen' is similar to the one between 'Man' and 'Woman'.

$$\vec{King} - \vec{Man} + \vec{Woman} \approx \vec{Queen}$$

Visualizing a Relationship

The arrow from "Man" to "King" represents the concept of "royalty". A parallel arrow from "Woman" points directly to "Queen".

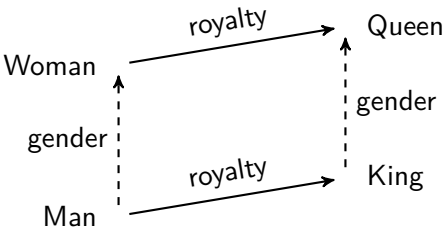


Figure: Vector arithmetic captures abstract relationships.

How Are Embeddings Obtained?

These vectors are not created by hand. They are **learned automatically** by algorithms that process billions of words from books, articles, and websites.

How Are Embeddings Obtained?

These vectors are not created by hand. They are **learned automatically** by algorithms that process billions of words from books, articles, and websites.

The algorithms

- Algorithms like **Word2Vec**, **GloVe**, and **FastText** learn embeddings by analyzing the context in which words appear.

How Are Embeddings Obtained?

These vectors are not created by hand. They are **learned automatically** by algorithms that process billions of words from books, articles, and websites.

The algorithms

- Algorithms like **Word2Vec**, **GloVe**, and **FastText** learn embeddings by analyzing the context in which words appear.
- The core idea is: *"A word is known by the company it keeps."*

Famous Ways of Learning Word Embeddings

Famous Ways of Learning Word Embeddings

- **Foundational Algorithms (e.g., Word2Vec, GloVe):**
  - These algorithms generate a single, fixed vector for each word. This is called a **static embedding**.

Famous Ways of Learning Word Embeddings

- **Foundational Algorithms (e.g., Word2Vec, GloVe):**
  - These algorithms generate a single, fixed vector for each word. This is called a **static embedding**.
  - The word "bank" has the same vector in "river bank" and "money bank".

Famous Ways of Learning Word Embeddings

- **Foundational Algorithms (e.g., Word2Vec, GloVe):**
  - These algorithms generate a single, fixed vector for each word. This is called a **static embedding**.
  - The word "bank" has the same vector in "river bank" and "money bank".
  - They learn by looking at word co-occurrence statistics in large text corpora.

Famous Ways of Learning Word Embeddings

- **Foundational Algorithms (e.g., Word2Vec, GloVe):**
  - These algorithms generate a single, fixed vector for each word. This is called a **static embedding**.
  - The word "bank" has the same vector in "river bank" and "money bank".
  - They learn by looking at word co-occurrence statistics in large text corpora.
- **Modern Models (e.g., BERT, GPT):**
  - These models generate **contextual embeddings**. The vector for a word changes based on the sentence it's in.

Famous Ways of Learning Word Embeddings

- **Foundational Algorithms (e.g., Word2Vec, GloVe):**
  - These algorithms generate a single, fixed vector for each word. This is called a **static embedding**.
  - The word "bank" has the same vector in "river bank" and "money bank".
  - They learn by looking at word co-occurrence statistics in large text corpora.
- **Modern Models (e.g., BERT, GPT):**
  - These models generate **contextual embeddings**. The vector for a word changes based on the sentence it's in.
  - **BERT** (from Google) reads the entire sentence at once (bidirectionally) to create a deeply contextualized embedding for each word.

Famous Ways of Learning Word Embeddings

- **Foundational Algorithms (e.g., Word2Vec, GloVe):**
  - These algorithms generate a single, fixed vector for each word. This is called a **static embedding**.
  - The word "bank" has the same vector in "river bank" and "money bank".
  - They learn by looking at word co-occurrence statistics in large text corpora.
- **Modern Models (e.g., BERT, GPT):**
  - These models generate **contextual embeddings**. The vector for a word changes based on the sentence it's in.
  - **BERT** (from Google) reads the entire sentence at once (bidirectionally) to create a deeply contextualized embedding for each word.
  - **GPT** (from OpenAI) reads text from left-to-right (auto-regressively) to generate its contextual embeddings, making it excellent for text generation.

A Simple Approach: Learn from Scratch

A common method used within larger models is to:

1. **Initialize Randomly:** Assign a vector of random numbers to every word in your vocabulary.

A Simple Approach: Learn from Scratch

A common method used within larger models is to:

1. **Initialize Randomly:** Assign a vector of random numbers to every word in your vocabulary.

2. **Learn Jointly:** Train a neural network on a specific task (e.g., predict the next word). As the model learns, it simultaneously adjusts the word embedding vectors along with its other weights.

A Simple Approach: Learn from Scratch

A common method used within larger models is to:

1. **Initialize Randomly:** Assign a vector of random numbers to every word in your vocabulary.

2. **Learn Jointly:** Train a neural network on a specific task (e.g., predict the next word). As the model learns, it simultaneously adjusts the word embedding vectors along with its other weights.

The model automatically figures out that "cat" and "pet" should have similar vectors because doing so helps it make better predictions.

Summary: Key Takeaways

- Word embeddings turn **words into numbers** (vectors) for computers.
- These vectors capture the **meaning and relationships** between words.
- Words with similar meanings are located **close together** in the "meaning space."
- They are **learned automatically** from vast amounts of text data or trained from scratch for a specific task.

Embeddings are a fundamental pillar of modern Natural Language Processing (NLP).

The Attention Module

Self Attention, Multi-Head Attention and Positional embeddings

The RNN Bottleneck Problem

The Core Limitation: A Single Vector Bottleneck

In a classic Encoder-Decoder RNN for machine translation:

- The **encoder** RNN processes each input token sequentially, updating its hidden state

The RNN Bottleneck Problem

The Core Limitation: A Single Vector Bottleneck

In a classic Encoder-Decoder RNN for machine translation:

- The **encoder** RNN processes each input token sequentially, updating its hidden state
- After processing the entire input, the **final hidden state** becomes the "context vector"

The RNN Bottleneck Problem

The Core Limitation: A Single Vector Bottleneck

In a classic Encoder-Decoder RNN for machine translation:

- The **encoder** RNN processes each input token sequentially, updating its hidden state
- After processing the entire input, the **final hidden state** becomes the "context vector"
- The **decoder** RNN uses only this single context vector to generate the entire output sequence

The RNN Bottleneck Problem

The Core Limitation: A Single Vector Bottleneck

In a classic Encoder-Decoder RNN for machine translation:

- The **encoder** RNN processes each input token sequentially, updating its hidden state
- After processing the entire input, the **final hidden state** becomes the "context vector"
- The **decoder** RNN uses only this single context vector to generate the entire output sequence

The problem intensifies with longer inputs:

- As input length increases, more information must be compressed into the same fixed-size vector

The RNN Bottleneck Problem

The Core Limitation: A Single Vector Bottleneck

In a classic Encoder-Decoder RNN for machine translation:

- The **encoder** RNN processes each input token sequentially, updating its hidden state
- After processing the entire input, the **final hidden state** becomes the "context vector"
- The **decoder** RNN uses only this single context vector to generate the entire output sequence

The problem intensifies with longer inputs:

- As input length increases, more information must be compressed into the same fixed-size vector
- Early information gets progressively diluted or lost entirely during encoding

The RNN Bottleneck Problem

The Core Limitation: A Single Vector Bottleneck

In a classic Encoder-Decoder RNN for machine translation:

- The **encoder** RNN processes each input token sequentially, updating its hidden state
- After processing the entire input, the **final hidden state** becomes the "context vector"
- The **decoder** RNN uses only this single context vector to generate the entire output sequence

The problem intensifies with longer inputs:

- As input length increases, more information must be compressed into the same fixed-size vector
- Early information gets progressively diluted or lost entirely during encoding
- Performance degrades dramatically beyond certain input lengths (typically 30-40 tokens)

The RNN Bottleneck Problem

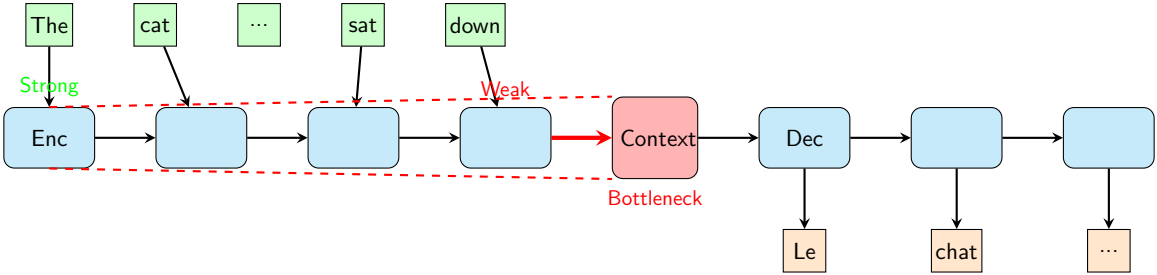


Figure: Single context vector forces all sequence information through a fixed-size bottleneck, causing information loss for longer sequences.

Other limitations of RNNs

Vanishing and Exploding Gradients

- Standard RNNs struggle with vanishing or exploding gradients.

Other limitations of RNNs

Vanishing and Exploding Gradients

- Standard RNNs struggle with vanishing or exploding gradients.
- This issue arises from the chain rule of differentiation applied repeatedly across many time steps.

Other limitations of RNNs

Vanishing and Exploding Gradients

- Standard RNNs struggle with vanishing or exploding gradients.
- This issue arises from the chain rule of differentiation applied repeatedly across many time steps.
- LSTMs and GRUs were designed to mitigate this problem, though they can still face challenges with extremely long sequences.

Other limitations of RNNs

Vanishing and Exploding Gradients

- Standard RNNs struggle with vanishing or exploding gradients.
- This issue arises from the chain rule of differentiation applied repeatedly across many time steps.
- LSTMs and GRUs were designed to mitigate this problem, though they can still face challenges with extremely long sequences.

Sequential Processing is slow

- RNNs process data one element at a time in a step-by-step loop.

Other limitations of RNNs

Vanishing and Exploding Gradients

- Standard RNNs struggle with vanishing or exploding gradients.
- This issue arises from the chain rule of differentiation applied repeatedly across many time steps.
- LSTMs and GRUs were designed to mitigate this problem, though they can still face challenges with extremely long sequences.

Sequential Processing is slow

- RNNs process data one element at a time in a step-by-step loop.
- This inherent sequential structure prevents parallel processing, making them computationally inefficient.

Other limitations of RNNs

Vanishing and Exploding Gradients

- Standard RNNs struggle with vanishing or exploding gradients.
- This issue arises from the chain rule of differentiation applied repeatedly across many time steps.
- LSTMs and GRUs were designed to mitigate this problem, though they can still face challenges with extremely long sequences.

Sequential Processing is slow

- RNNs process data one element at a time in a step-by-step loop.
- This inherent sequential structure prevents parallel processing, making them computationally inefficient.
- As a result, training RNNs on long sequences is extremely time-consuming.

Introduction to Attention

A Paradigm Shift from RNNs

- **The Old Guard:** For years, Recurrent Neural Networks (RNNs), particularly LSTMs, were the state-of-the-art for most Natural Language Processing (NLP) tasks.

Introduction to Attention

A Paradigm Shift from RNNs

- **The Old Guard:** For years, Recurrent Neural Networks (RNNs), particularly LSTMs, were the state-of-the-art for most Natural Language Processing (NLP) tasks.
- **Attention as an Enhancement:** The **attention mechanism** allowed the model to dynamically "attend" to different parts of the input sequence at each step of the output generation, overcoming the fixed-vector limitation.

Introduction to Attention

A Paradigm Shift from RNNs

- **The Old Guard:** For years, Recurrent Neural Networks (RNNs), particularly LSTMs, were the state-of-the-art for most Natural Language Processing (NLP) tasks.
- **Attention as an Enhancement:** The **attention mechanism** allowed the model to dynamically "attend" to different parts of the input sequence at each step of the output generation, overcoming the fixed-vector limitation.
- **The Transformer Revolution:** The game-changing "Attention Is All You Need" paper introduced the **Transformer**, an architecture that solely on attention mechanisms to model relationships between tokens.

Introduction to Attention

A Paradigm Shift from RNNs

- **The Old Guard:** For years, Recurrent Neural Networks (RNNs), particularly LSTMs, were the state-of-the-art for most Natural Language Processing (NLP) tasks.
- **Attention as an Enhancement:** The **attention mechanism** allowed the model to dynamically "attend" to different parts of the input sequence at each step of the output generation, overcoming the fixed-vector limitation.
- **The Transformer Revolution:** The game-changing "Attention Is All You Need" paper introduced the **Transformer**, an architecture that solely on attention mechanisms to model relationships between tokens.
- **Modern Dominance:** Today, Transformers are the foundation for nearly all state-of-the-art models in NLP (e.g., BERT, GPT) and are now a dominant force in computer vision (Vision Transformer) and beyond.

The Intuition Behind the Attention Mechanism

The Human Analogy: How do we translate?

When a human translates a sentence, they don't read the whole sentence, memorize it, and then write the translation. Instead, they focus on different parts of the source sentence as they generate each part of the target sentence.

The Intuition Behind the Attention Mechanism

The Human Analogy: How do we translate?

When a human translates a sentence, they don't read the whole sentence, memorize it, and then write the translation. Instead, they focus on different parts of the source sentence as they generate each part of the target sentence.

Example Sentence

"The cat is sleeping on the mat."

1. When generating the Hindi translation *"billi chataai par so rahi hai"* (cat is sleeping on mat), to produce the word "billi" (cat), a translator would "pay attention" to the word "cat".
2. Similarly, for the Tamil translation *"poonai paayil thoongukiradhu"* (cat is sleeping on mat), to produce the word "poonai" (cat), the translator focuses attention on the word "cat".

The Attention in AI

The Core Idea of Attention in AI

Instead of a single bottleneck vector, the attention mechanism allows the decoder to directly **look back at the entire source sentence** at every step of the output generation.

The Attention in AI

The Core Idea of Attention in AI

Instead of a single bottleneck vector, the attention mechanism allows the decoder to directly **look back at the entire source sentence** at every step of the output generation.

- It learns to compute a set of **attention weights** for each output word.

The Attention in AI

The Core Idea of Attention in AI

Instead of a single bottleneck vector, the attention mechanism allows the decoder to directly **look back at the entire source sentence** at every step of the output generation.

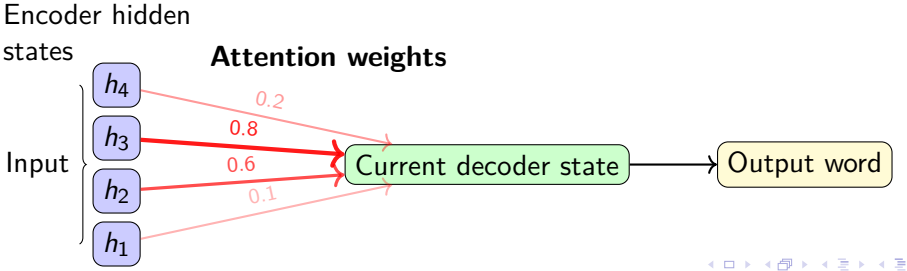
- It learns to compute a set of **attention weights** for each output word.
- These weights determine how much "attention" to pay to each input word when generating the current output word.

The Attention in AI

The Core Idea of Attention in AI

Instead of a single bottleneck vector, the attention mechanism allows the decoder to directly **look back at the entire source sentence** at every step of the output generation.

- It learns to compute a set of **attention weights** for each output word.
- These weights determine how much "attention" to pay to each input word when generating the current output word.



The Attention Mechanism: Queries, Keys, and Values

Attention can be described as a process of mapping a **Query** and a set of **Key-Value** pairs to an output. This is the core of modern "self-attention".

The Attention Mechanism: Queries, Keys, and Values

Attention can be described as a process of mapping a **Query** and a set of **Key-Value** pairs to an output. This is the core of modern "self-attention".

The Cookbook Analogy

Imagine you want to cook a dish (**Query**). You have a cookbook with many recipes.

1. **Compare Query to Keys:** You compare your desired dish (*Query*) to the title of each recipe (*Key*) to find how relevant each recipe is. This gives you a **score**.

The Attention Mechanism: Queries, Keys, and Values

Attention can be described as a process of mapping a **Query** and a set of **Key-Value** pairs to an output. This is the core of modern "self-attention".

The Cookbook Analogy

Imagine you want to cook a dish (**Query**). You have a cookbook with many recipes.

1. **Compare Query to Keys:** You compare your desired dish (*Query*) to the title of each recipe (*Key*) to find how relevant each recipe is. This gives you a **score**.
2. **Normalize Scores:** You turn these scores into probabilities (e.g., using softmax). This tells you what percentage of your focus should be on each recipe.

The Attention Mechanism: Queries, Keys, and Values

Attention can be described as a process of mapping a **Query** and a set of **Key-Value** pairs to an output. This is the core of modern "self-attention".

The Cookbook Analogy

Imagine you want to cook a dish (**Query**). You have a cookbook with many recipes.

1. **Compare Query to Keys:** You compare your desired dish (*Query*) to the title of each recipe (*Key*) to find how relevant each recipe is. This gives you a **score**.
2. **Normalize Scores:** You turn these scores into probabilities (e.g., using softmax). This tells you what percentage of your focus should be on each recipe.
3. **Get Weighted Values:** You create your final dish (*Output*) by taking a weighted blend of the actual recipe contents (*Value*) based on your focus percentages.

The Attention Mechanism: Queries, Keys and Values

The Technical Implementation

In an Attention-based model, each word in a sentence is disentagled into three vectors:

The Attention Mechanism: Queries, Keys and Values

The Technical Implementation

In an Attention-based model, each word in a sentence is disentagled into three vectors:

- **Query (Q):** A vector representing the current word asking, "Who should I pay attention to?"

The Attention Mechanism: Queries, Keys and Values

The Technical Implementation

In an Attention-based model, each word in a sentence is disentagled into three vectors:

- **Query (Q):** A vector representing the current word asking, "Who should I pay attention to?"
- **Key (K):** A vector for each word in the sentence saying, "This is what I am."

The Attention Mechanism: Queries, Keys and Values

The Technical Implementation

In an Attention-based model, each word in a sentence is disentagled into three vectors:

- **Query (Q):** A vector representing the current word asking, "Who should I pay attention to?"
- **Key (K):** A vector for each word in the sentence saying, "This is what I am."
- **Value (V):** A vector for each word in the sentence saying, "This is what I have to offer."

These Q, K, and V vectors are learned projections of the original word embeddings.

The Core Idea: Attention Pooling

The fundamental operation of an attention mechanism is often called **attention pooling**.

The Core Idea: Attention Pooling

The fundamental operation of an attention mechanism is often called **attention pooling**.

Given a query $\mathbf{q}$ and a database of key-value pairs $\mathcal{D} = \{(\mathbf{k}_1, \mathbf{v}_1), \ldots, (\mathbf{k}_m, \mathbf{v}_m)\}$, the output is a weighted sum of the values:

$$\text{Attention}(\mathbf{q}, \mathcal{D}) \stackrel{\text{def}}{=} \sum_{i=1}^{m} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i \tag{1}$$

The Core Idea: Attention Pooling

The fundamental operation of an attention mechanism is often called **attention pooling**.

Given a query $\mathbf{q}$ and a database of key-value pairs $\mathcal{D} = \{(\mathbf{k}_1, \mathbf{v}_1), \ldots, (\mathbf{k}_m, \mathbf{v}_m)\}$, the output is a weighted sum of the values:

$$\text{Attention}(\mathbf{q}, \mathcal{D}) \stackrel{\text{def}}{=} \sum_{i=1}^{m} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i \tag{1}$$

- The scalar weights $\alpha(\mathbf{q}, \mathbf{k}_i)$ are the **attention weights**.

The Core Idea: Attention Pooling

The fundamental operation of an attention mechanism is often called **attention pooling**.

Given a query $\mathbf{q}$ and a database of key-value pairs $\mathcal{D} = \{(\mathbf{k}_1, \mathbf{v}_1), \ldots, (\mathbf{k}_m, \mathbf{v}_m)\}$, the output is a weighted sum of the values:

$$\text{Attention}(\mathbf{q}, \mathcal{D}) \stackrel{\text{def}}{=} \sum_{i=1}^{m} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i \tag{1}$$

- The scalar weights $\alpha(\mathbf{q}, \mathbf{k}_i)$ are the **attention weights**.
- The mechanism "pays attention" to the values $\mathbf{v}_i$ for which the corresponding weight α is significant.

Properties of Attention Weights

The behavior of the attention mechanism depends entirely on the properties of its weights:

Properties of Attention Weights

The behavior of the attention mechanism depends entirely on the properties of its weights:

- **Non-Negative Weights:** If all $\alpha(\mathbf{q}, \mathbf{k}_i) \geq 0$, the output is contained in the convex cone spanned by the values $\{\mathbf{v}_i\}$.

Properties of Attention Weights

The behavior of the attention mechanism depends entirely on the properties of its weights:

- **Non-Negative Weights:** If all $\alpha(\mathbf{q}, \mathbf{k}_i) \geq 0$, the output is contained in the convex cone spanned by the values $\{\mathbf{v}_i\}$.

- **Convex Combination (Most Common):** The weights form a probability distribution, where $\sum_i \alpha(\mathbf{q}, \mathbf{k}_i) = 1$ and $\alpha(\mathbf{q}, \mathbf{k}_i) \geq 0$. This is the standard setting in deep learning.

Properties of Attention Weights

The behavior of the attention mechanism depends entirely on the properties of its weights:

- **Non-Negative Weights:** If all $\alpha(\mathbf{q}, \mathbf{k}_i) \geq 0$, the output is contained in the convex cone spanned by the values $\{\mathbf{v}_i\}$.
- **Convex Combination (Most Common):** The weights form a probability distribution, where $\sum_i \alpha(\mathbf{q}, \mathbf{k}_i) = 1$ and $\alpha(\mathbf{q}, \mathbf{k}_i) \geq 0$. This is the standard setting in deep learning.
- **Hard Attention:** Exactly one weight is 1, and all others are 0. This is analogous to a traditional database query, selecting a single value.

Properties of Attention Weights

The behavior of the attention mechanism depends entirely on the properties of its weights:

- **Non-Negative Weights:** If all $\alpha(\mathbf{q}, \mathbf{k}_i) \geq 0$, the output is contained in the convex cone spanned by the values $\{\mathbf{v}_i\}$.
- **Convex Combination (Most Common):** The weights form a probability distribution, where $\sum_i \alpha(\mathbf{q}, \mathbf{k}_i) = 1$ and $\alpha(\mathbf{q}, \mathbf{k}_i) \geq 0$. This is the standard setting in deep learning.
- **Hard Attention:** Exactly one weight is 1, and all others are 0. This is analogous to a traditional database query, selecting a single value.
- **Average Pooling:** All weights are equal, i.e., $\alpha(\mathbf{q}, \mathbf{k}_i) = \frac{1}{m}$. This simply averages all values in the database.

How to Calculate Attention Weights

To ensure weights are non-negative and sum to 1 (a convex combination), we use a two-step process:

How to Calculate Attention Weights

To ensure weights are non-negative and sum to 1 (a convex combination), we use a two-step process:

1. First, we choose **any scoring function**, $a(\mathbf{q}, \mathbf{k})$, that measures the similarity or relevance between the query and a key.

How to Calculate Attention Weights

To ensure weights are non-negative and sum to 1 (a convex combination), we use a two-step process:

1. First, we choose **any scoring function**, $a(\mathbf{q}, \mathbf{k})$, that measures the similarity or relevance between the query and a key.

2. Second, we apply the **softmax** operation to these scores to transform them into valid attention weights.

How to Calculate Attention Weights

To ensure weights are non-negative and sum to 1 (a convex combination), we use a two-step process:

1. First, we choose **any scoring function**, $a(\mathbf{q}, \mathbf{k})$, that measures the similarity or relevance between the query and a key.

2. Second, we apply the **softmax** operation to these scores to transform them into valid attention weights.

The formula for the attention weight is:

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \text{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^{m} \exp(a(\mathbf{q}, \mathbf{k}_j))} \tag{2}$$

How to Calculate Attention Weights

To ensure weights are non-negative and sum to 1 (a convex combination), we use a two-step process:

1. First, we choose **any scoring function**, $a(\mathbf{q}, \mathbf{k})$, that measures the similarity or relevance between the query and a key.

2. Second, we apply the **softmax** operation to these scores to transform them into valid attention weights.

The formula for the attention weight is:

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \text{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^{m} \exp(a(\mathbf{q}, \mathbf{k}_j))} \tag{2}$$

This approach is ideal for deep learning because it is **differentiable** and its **gradient never vanishes**.

Attention Mechanism: A Visual Walkthrough

1. Input Words

Thinking

Machines

Figure: A step-by-step visualization of the self-attention mechanism for the first token.

Attention Mechanism: A Visual Walkthrough

1. Input Words 2. Word Embeddings

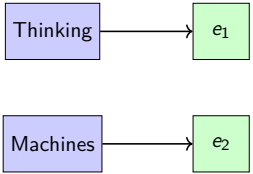


Figure: A step-by-step visualization of the self-attention mechanism for the first token.

Attention Mechanism: A Visual Walkthrough

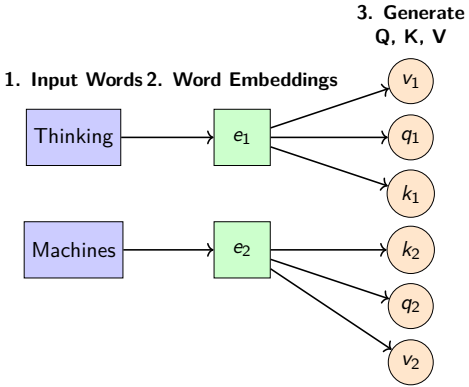


Figure: A step-by-step visualization of the self-attention mechanism for the first token.

Attention Mechanism: A Visual Walkthrough

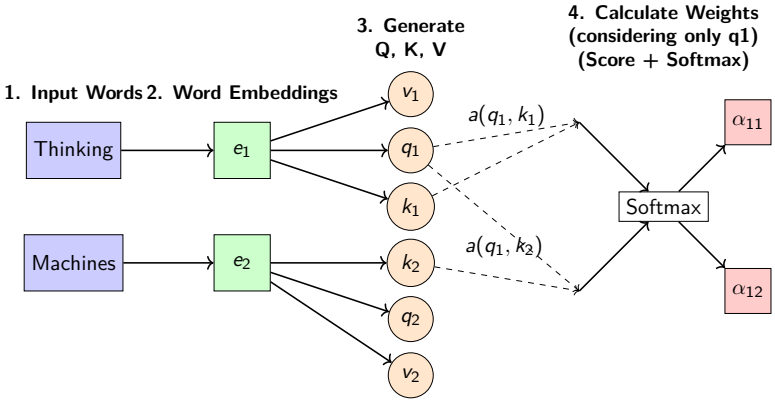


Figure: A step-by-step visualization of the self-attention mechanism for the first token.

Attention Mechanism: A Visual Walkthrough

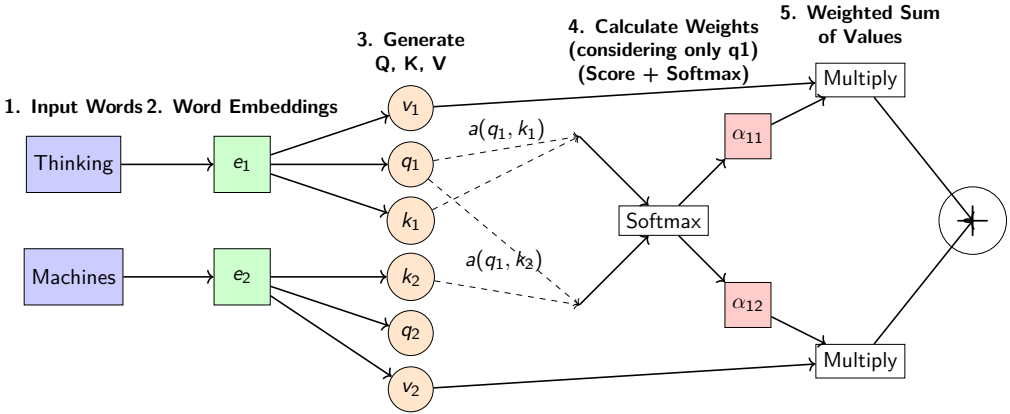


Figure: A step-by-step visualization of the self-attention mechanism for the first token.

Attention Mechanism: A Visual Walkthrough

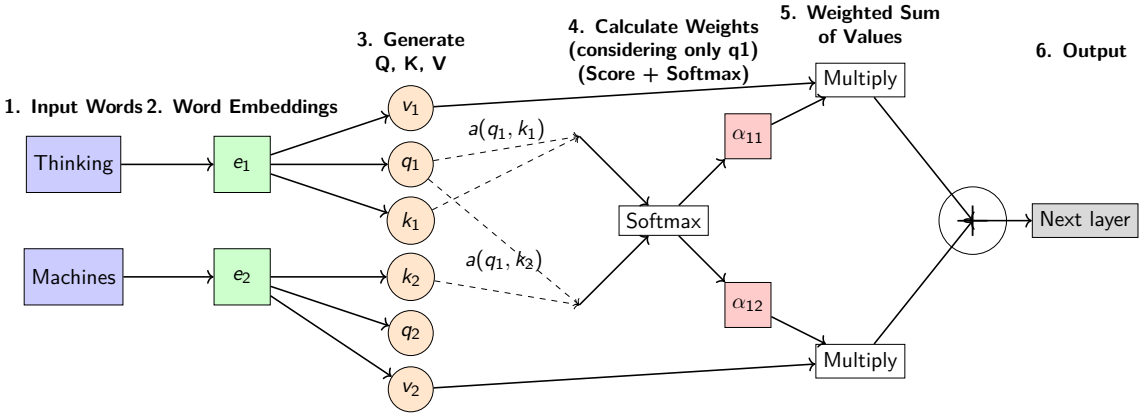


Figure: A step-by-step visualization of the self-attention mechanism for the first token.

Similarity Functions (Kernels)

- The similarity between a query q and a key k can be defined by various kernel functions $\alpha(q, k)$.
- Here are some common examples:

Gaussian Kernel

$$\alpha(q, k) = \exp\left(-\frac{1}{2}\|q - k\|^2\right)$$

Boxcar Kernel

$$\alpha(q, k) = \begin{cases} 1 & \text{if } \|q - k\| \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

Epanechnikov Kernel

$$\alpha(q, k) = \max(0, 1 - \|q - k\|)$$

Similarity Functions Visualization

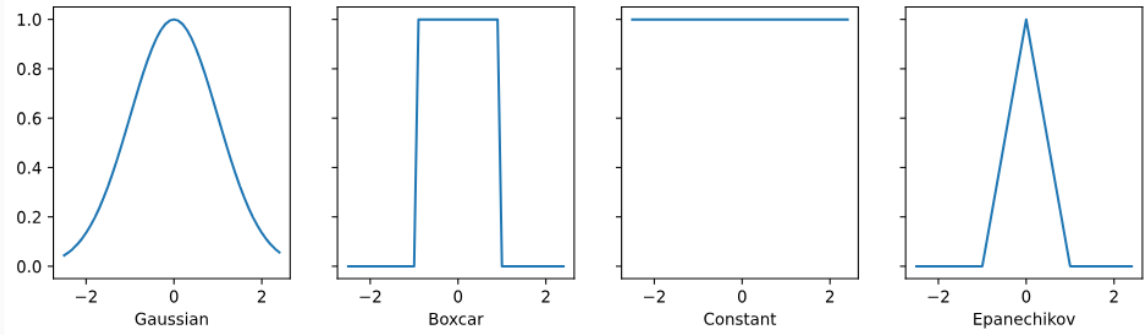


Figure: Similarity Functions

Derivation from Gaussian Kernel

- We can derive dot product attention by reviewing the Gaussian kernel *without* exponentiation.

Derivation from Gaussian Kernel

- We can derive dot product attention by reviewing the Gaussian kernel *without* exponentiation.
- Expanding the squared norm yields the following scoring function:

$$a(\mathbf{q}, \mathbf{k}_i) = -\frac{1}{2}\|\mathbf{q} - \mathbf{k}_i\|^2 = \mathbf{q}^\top \mathbf{k}_i - \frac{1}{2}\|\mathbf{k}_i\|^2 - \frac{1}{2}\|\mathbf{q}\|^2 \tag{1}$$

Derivation from Gaussian Kernel

- We can derive dot product attention by reviewing the Gaussian kernel *without* exponentiation.
- Expanding the squared norm yields the following scoring function:

$$a(\mathbf{q}, \mathbf{k}_i) = -\frac{1}{2}\|\mathbf{q} - \mathbf{k}_i\|^2 = \mathbf{q}^\top \mathbf{k}_i - \frac{1}{2}\|\mathbf{k}_i\|^2 - \frac{1}{2}\|\mathbf{q}\|^2 \tag{1}$$

- We can simplify this expression based on how attention weights are normalized.

Simplifying the Scoring Function

Looking at the terms in Equation (1):

Simplifying the Scoring Function

Looking at the terms in Equation (1):

1. **The q term:** The term $-\frac{1}{2}\|\mathbf{q}\|^2$ depends only on $\mathbf{q}$. It is identical for all $(\mathbf{q}, \mathbf{k}_i)$ pairs.
   - When normalizing attention weights to sum to 1 (e.g., using softmax), this term cancels out entirely.

Simplifying the Scoring Function

Looking at the terms in Equation (1):

1. **The q term:** The term $-\frac{1}{2}\|\mathbf{q}\|^2$ depends only on $\mathbf{q}$. It is identical for all $(\mathbf{q}, \mathbf{k}_i)$ pairs.
   - When normalizing attention weights to sum to 1 (e.g., using softmax), this term cancels out entirely.
2. **The $\mathbf{k}_i$ term:** The term $-\frac{1}{2}\|\mathbf{k}_i\|^2$ can be dropped if the keys have constant norms.
   - This is often the case, such as when keys are generated by layer normalization.

Simplifying the Scoring Function

Looking at the terms in Equation (1):

1. **The q term:** The term $-\frac{1}{2}\|\mathbf{q}\|^2$ depends only on $\mathbf{q}$. It is identical for all $(\mathbf{q}, \mathbf{k}_i)$ pairs.
   - When normalizing attention weights to sum to 1 (e.g., using softmax), this term cancels out entirely.
2. **The $\mathbf{k}_i$ term:** The term $-\frac{1}{2}\|\mathbf{k}_i\|^2$ can be dropped if the keys have constant norms.
   - This is often the case, such as when keys are generated by layer normalization.

After dropping these terms, we are left with the dot product $\mathbf{q}^\top \mathbf{k}_i$.

Scaled Dot Product Attention

- We must keep the order of magnitude of the arguments under control.

Scaled Dot Product Attention

- We must keep the order of magnitude of the arguments under control.
- Assume query $\mathbf{q} \in \mathbb{R}^d$ and key $\mathbf{k}_i \in \mathbb{R}^d$ are independent variables with zero mean and unit variance.

Scaled Dot Product Attention

- We must keep the order of magnitude of the arguments under control.
- Assume query $\mathbf{q} \in \mathbb{R}^d$ and key $\mathbf{k}_i \in \mathbb{R}^d$ are independent variables with zero mean and unit variance.
- Their dot product has zero mean, but a variance of d.

Scaled Dot Product Attention

- We must keep the order of magnitude of the arguments under control.
- Assume query $\mathbf{q} \in \mathbb{R}^d$ and key $\mathbf{k}_i \in \mathbb{R}^d$ are independent variables with zero mean and unit variance.
- Their dot product has zero mean, but a variance of d.
- To ensure the variance remains 1 regardless of vector length, we use **scaled dot product attention** by rescaling by $1/\sqrt{d}$.

Scaled Dot Product Attention

- We must keep the order of magnitude of the arguments under control.
- Assume query $\mathbf{q} \in \mathbb{R}^d$ and key $\mathbf{k}_i \in \mathbb{R}^d$ are independent variables with zero mean and unit variance.
- Their dot product has zero mean, but a variance of d.
- To ensure the variance remains 1 regardless of vector length, we use **scaled dot product attention** by rescaling by $1/\sqrt{d}$.

Scoring Function ("Attention is all you need", Vaswani et al., 2017)

$$a(\mathbf{q}, \mathbf{k}_i) = \mathbf{q}^\top \mathbf{k}_i / \sqrt{d} \tag{2}$$

Final Attention Weights (Softmax)

- The scoring function $a(\mathbf{q}, \mathbf{k}_i)$ still needs to be normalized to obtain the final attention weights α.

Final Attention Weights (Softmax)

- The scoring function $a(\mathbf{q}, \mathbf{k}_i)$ still needs to be normalized to obtain the final attention weights α.
- Hence we apply the softmax operation.

Final Attention Weights (Softmax)

- The scoring function $a(\mathbf{q}, \mathbf{k}_i)$ still needs to be normalized to obtain the final attention weights α.
- Hence we apply the softmax operation.

Normalized Attention Weights

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \text{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(\mathbf{q}^\top \mathbf{k}_i / \sqrt{d})}{\sum_{j=1} \exp(\mathbf{q}^\top \mathbf{k}_j / \sqrt{d})} \tag{3}$$

Final Attention Weights (Softmax)

- The scoring function $a(\mathbf{q}, \mathbf{k}_i)$ still needs to be normalized to obtain the final attention weights α.
- Hence we apply the softmax operation.

Normalized Attention Weights

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \text{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(\mathbf{q}^\top \mathbf{k}_i / \sqrt{d})}{\sum_{j=1} \exp(\mathbf{q}^\top \mathbf{k}_j / \sqrt{d})} \tag{3}$$

Most popular attention mechanisms use this softmax approach.

Scaled Dot Product Attention (Final Form)

Matrix Definitions

Let's consider:

- **Queries** $\mathbf{Q} \in \mathbb{R}^{n \times d}$ (n queries of length d)
- **Keys** $\mathbf{K} \in \mathbb{R}^{m \times d}$ (m keys of length d)
- **Values** $\mathbf{V} \in \mathbb{R}^{m \times v}$ (m values of length v)

Scaled Dot Product Attention (Final Form)

Matrix Definitions

Let's consider:

- **Queries** $\mathbf{Q} \in \mathbb{R}^{n \times d}$ (n queries of length d)
- **Keys** $\mathbf{K} \in \mathbb{R}^{m \times d}$ (m keys of length d)
- **Values** $\mathbf{V} \in \mathbb{R}^{m \times v}$ (m values of length v)

The Scaled Dot Product Attention Formula

The attention output can be written concisely as:

$$\text{Output} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V} \in \mathbb{R}^{n \times v} \tag{4}$$

Scaled Dot Product Attention (Final Form)

Matrix Definitions

Let's consider:

- **Queries** $\mathbf{Q} \in \mathbb{R}^{n \times d}$ (n queries of length d)
- **Keys** $\mathbf{K} \in \mathbb{R}^{m \times d}$ (m keys of length d)
- **Values** $\mathbf{V} \in \mathbb{R}^{m \times v}$ (m values of length v)

The Scaled Dot Product Attention Formula

The attention output can be written concisely as:

$$\text{Output} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V} \in \mathbb{R}^{n \times v} \tag{4}$$

The result is a weighted sum of the values $\mathbf{V}$, where the weights are determined by the scaled dot-product similarity between queries $\mathbf{Q}$ and keys $\mathbf{K}$.

Introducing the Two Forms of Attention

1. Self-Attention

Self-attention relates different positions of a **single sequence or single modality**(only text, only image, only audio, etc) to compute a representation of that same sequence. It allows the model to understand the internal relationships and context within the input.

Introducing the Two Forms of Attention

1. Self-Attention

Self-attention relates different positions of a **single sequence or single modality**(only text, only image, only audio, etc) to compute a representation of that same sequence. It allows the model to understand the internal relationships and context within the input.

- **Core Idea:** "For each token in this sequence, how relevant are the other tokens in the *same* sequence?"

Introducing the Two Forms of Attention

1. Self-Attention

Self-attention relates different positions of a **single sequence or single modality**(only text, only image, only audio, etc) to compute a representation of that same sequence. It allows the model to understand the internal relationships and context within the input.

- **Core Idea:** "For each token in this sequence, how relevant are the other tokens in the *same* sequence?"
- **Vector Source:** The Query (**Q**), Key (**K**), and Value (**V**) vectors are all derived from the **same input sequence**.

Introducing the Two Forms of Attention

1. Self-Attention

Self-attention relates different positions of a **single sequence or single modality**(only text, only image, only audio, etc) to compute a representation of that same sequence. It allows the model to understand the internal relationships and context within the input.

- **Core Idea:** "For each token in this sequence, how relevant are the other tokens in the *same* sequence?"
- **Vector Source:** The Query (**Q**), Key (**K**), and Value (**V**) vectors are all derived from the **same input sequence**.
- **Primary Use:** Building rich, context-aware representations. It's the core component of encoders like BERT.

Introducing the Two Forms of Attention

2. Cross-Attention

Cross-attention relates the positions of **two different sequences**. It's the mechanism that connects an encoder and a decoder in sequence-to-sequence tasks.

Introducing the Two Forms of Attention

2. Cross-Attention

Cross-attention relates the positions of **two different sequences**. It's the mechanism that connects an encoder and a decoder in sequence-to-sequence tasks.

- **Core Idea:** "While generating a word in the target sentence, which words from the *source* sentence are most relevant?"

Introducing the Two Forms of Attention

2. Cross-Attention

Cross-attention relates the positions of **two different sequences**. It's the mechanism that connects an encoder and a decoder in sequence-to-sequence tasks.

- **Core Idea:** "While generating a word in the target sentence, which words from the *source* sentence are most relevant?"
- **Vector Source:** The Query (**Q**) vectors come from one sequence (e.g., the decoder), while the Key (**K**) and Value (**V**) vectors come from a **second sequence** (e.g., the encoder).

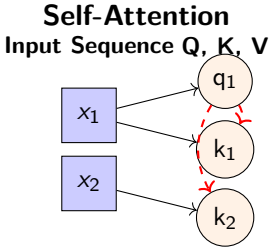
Introducing the Two Forms of Attention

2. Cross-Attention

Cross-attention relates the positions of **two different sequences**. It's the mechanism that connects an encoder and a decoder in sequence-to-sequence tasks.

- **Core Idea:** "While generating a word in the target sentence, which words from the *source* sentence are most relevant?"
- **Vector Source:** The Query (**Q**) vectors come from one sequence (e.g., the decoder), while the Key (**K**) and Value (**V**) vectors come from a **second sequence** (e.g., the encoder).
- **Primary Use:** Aligning and fusing information from two distinct sequences, crucial for machine translation, text summarization, etc.

Key Differences: Self-Attention vs. Cross-Attention

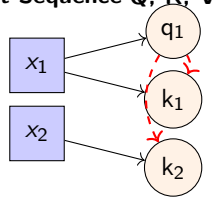


- **Source:** Q, K, and V matrices originate from a **single sequence**.
- **Purpose:** To learn the internal dependencies and structure of one sequence.

Key Differences: Self-Attention vs. Cross-Attention

Self-Attention

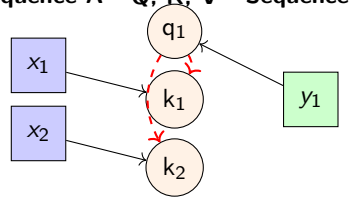
Input Sequence Q, K, V



- **Source:** Q, K, and V matrices originate from a **single sequence**.
- **Purpose:** To learn the internal dependencies and structure of one sequence.

Cross-Attention

Sequence A **Q, K, V** **Sequence B**



- **Source:** Q comes from one sequence, while K and V come from a **second sequence**.
- **Purpose:** To align and integrate information from two different sequences.

The Masked Softmax Operation

The Problem: Variable Sequence Lengths

- Attention is often applied to sequences (e.g., sentences in NLP).

The Masked Softmax Operation

The Problem: Variable Sequence Lengths

- Attention is often applied to sequences (e.g., sentences in NLP).
- To process sequences in a minibatch, shorter sequences must be padded with special tokens to match the length of the longest one.

The Masked Softmax Operation

The Problem: Variable Sequence Lengths

- Attention is often applied to sequences (e.g., sentences in NLP).
- To process sequences in a minibatch, shorter sequences must be padded with special tokens to match the length of the longest one.
- **Example:**

```
Dive into Deep Learning
Learn to code <blank>
Hello world <blank> <blank>
```

The Masked Softmax Operation

The Problem: Variable Sequence Lengths

- Attention is often applied to sequences (e.g., sentences in NLP).
- To process sequences in a minibatch, shorter sequences must be padded with special tokens to match the length of the longest one.
- **Example:**

```
Dive into Deep Learning
Learn to code <blank>
Hello world <blank> <blank>
```

- These padding tokens ('<blank>') have no meaning and should not contribute to the attention output.

The Masked Softmax Operation

The Solution: Masking

We need to filter out the padded tokens before the softmax calculation. This is called the **masked softmax operation**.

The Masked Softmax Operation

The Solution: Masking

We need to filter out the padded tokens before the softmax calculation. This is called the **masked softmax operation**.

1. **Goal:** Limit the attention summation to only the valid tokens in the sequence (i.e., ignore the padded parts).

The Masked Softmax Operation

The Solution: Masking

We need to filter out the padded tokens before the softmax calculation. This is called the **masked softmax operation**.

1. **Goal:** Limit the attention summation to only the valid tokens in the sequence (i.e., ignore the padded parts).
2. **Implementation "Cheat":** Instead of using conditional logic (which is slow on GPUs), we take the scores for the padded positions and set them to a very large negative number (e.g., -10^6).

The Masked Softmax Operation

The Solution: Masking

We need to filter out the padded tokens before the softmax calculation. This is called the **masked softmax operation**.

1. **Goal:** Limit the attention summation to only the valid tokens in the sequence (i.e., ignore the padded parts).
2. **Implementation "Cheat":** Instead of using conditional logic (which is slow on GPUs), we take the scores for the padded positions and set them to a very large negative number (e.g., -10^6).
3. **Result:** When the softmax function is applied, the exponent of this large negative number becomes effectively zero ($\exp(-10^6) \approx 0$). This ensures the padded tokens have no influence on the final attention weights.

The Transformer Architecture

Self Attention, Multi-Head Attention and Positional embeddings

From RNNs + Attention to Transformers

What We've Built So Far

- RNNs to process sequences (one token at a time)
- Attention mechanisms to focus on relevant parts
- Self-attention and cross-attention
- Scaled dot-product attention

From RNNs + Attention to Transformers

What We've Built So Far

- RNNs to process sequences (one token at a time)
- Attention mechanisms to focus on relevant parts
- Self-attention and cross-attention
- Scaled dot-product attention

The Bottleneck

RNNs are **sequential** – we can't process the next word until we've processed the previous one. This makes training slow and long-range dependencies difficult.

From RNNs + Attention to Transformers

What We've Built So Far

- RNNs to process sequences (one token at a time)
- Attention mechanisms to focus on relevant parts
- Self-attention and cross-attention
- Scaled dot-product attention

The Bottleneck

RNNs are **sequential** – we can't process the next word until we've processed the previous one. This makes training slow and long-range dependencies difficult.

The Breakthrough (Vaswani et al., 2017)

"What if we threw away the RNN entirely and used **only attention**?"

Welcome to the **Transformer** – you already know its core mechanisms!

RNN-Based vs. Transformer-Based Models

Concept	RNN-Based Models	Transformer-Based Models
Core Computation	Sequential processing (one step at a time)	**Parallel** processing of all positions
Context Handling	Needs attention *added on top* to focus	**Self-Attention is the core** operation
Long-Range Dependencies	Hard to remember distant information (vanishing gradients)	**Direct interactions** between any two tokens
Computation Pattern	Recurrence $\rightarrow$ O(T) sequential depth	**Matrix operations** $\rightarrow$ O(1) parallel depth
Scalability	Slows down as sequences grow	**Scales well** with data + hardware

Impact of Transformers on Generative AI

Technical Breakthroughs

- **Scalability:** Billions $\rightarrow$ Trillions of parameters
- **Context:** 512 tokens $\rightarrow$ 1M+ tokens
- **Multimodality:** Text, images, video, audio, code
- **Emergent abilities:** Reasoning, planning, tool use

Real-World Impact

- Conversational AI assistants
- Code generation & debugging
- Content creation (text, art, video)
- Scientific discovery (protein folding, drug design)
- Translation, summarization, analysis

Impact of Transformers on Generative AI

Technical Breakthroughs

- **Scalability:** Billions $\rightarrow$ Trillions of parameters
- **Context:** 512 tokens $\rightarrow$ 1M+ tokens
- **Multimodality:** Text, images, video, audio, code
- **Emergent abilities:** Reasoning, planning, tool use

Real-World Impact

- Conversational AI assistants
- Code generation & debugging
- Content creation (text, art, video)
- Scientific discovery (protein folding, drug design)
- Translation, summarization, analysis

The Key Insight

One architecture + massive scale + diverse data = general intelligence

The Transformer Revolution in Generative AI

"Attention Is All You Need" (2017) changed everything

Year	Breakthrough
2018	BERT: Pre-training + Fine-tuning paradigm
2019	GPT-2: Large-scale text generation
2020	GPT-3: Few-shot learning, 175B parameters
2021	DALL-E: Text-to-image, Codex: Code generation
2022	ChatGPT: Conversational AI goes mainstream
2023-24	GPT-4, Claude, Gemini: Multimodal reasoning

The Transformer Revolution in Generative AI

"Attention Is All You Need" (2017) changed everything

Year	Breakthrough
2018	BERT: Pre-training + Fine-tuning paradigm
2019	GPT-2: Large-scale text generation
2020	GPT-3: Few-shot learning, 175B parameters
2021	DALL-E: Text-to-image, Codex: Code generation
2022	ChatGPT: Conversational AI goes mainstream
2023-24	GPT-4, Claude, Gemini: Multimodal reasoning

The Common Thread

All powered by the **same core architecture** you're learning today!

The Original Transformer Architecture

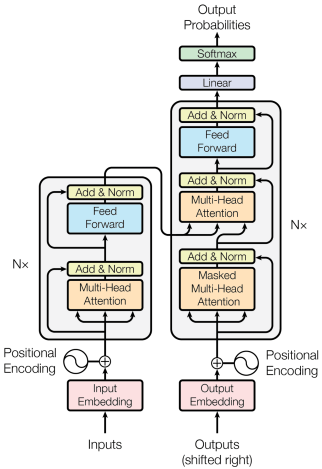
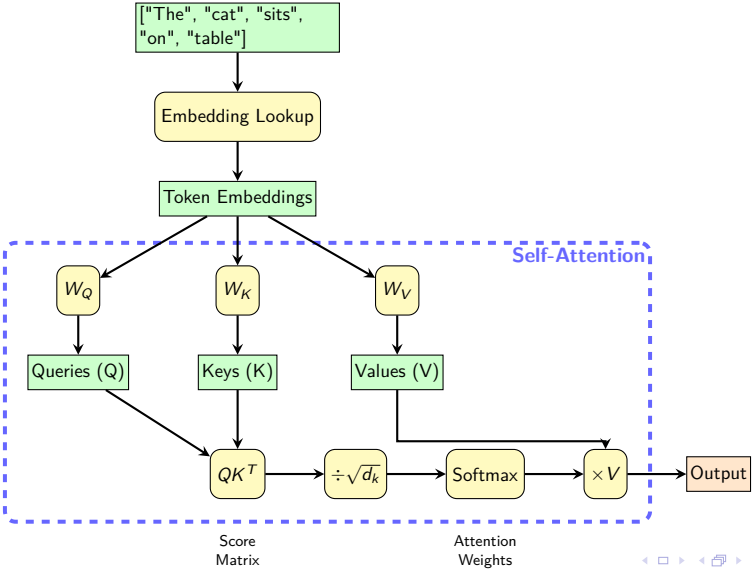


Figure: Transformer Architecture. (source: "Attention is All you need" paper)

Self-Attention Visualization



The Core Engine: Multi-Head Attention

The Problem: Sentences Have Many Relationships

In the sentence "The animal didn't cross the street because it was too tired", what does "it" refer to? The animal or the street? A single attention mechanism might get confused.

The Core Engine: Multi-Head Attention

The Problem: Sentences Have Many Relationships

In the sentence "The animal didn't cross the street because it was too tired", what does "it" refer to? The animal or the street? A single attention mechanism might get confused.

The Solution: Multi-Head Attention

Instead of one attention mechanism, we use multiple "heads" that work in parallel.

Multi-Head Attention: A Committee of Experts

Analogy: A Committee of Experts

Imagine each attention "head" is an expert focusing on a different type of relationship:

Multi-Head Attention: A Committee of Experts

Analogy: A Committee of Experts

Imagine each attention "head" is an expert focusing on a different type of relationship:

- **Expert 1 (Syntax):** Focuses on subject-verb agreement.
- **Expert 2 (Semantics):** Focuses on which words have similar meanings.
- **Expert 3 (Pronouns):** Focuses on linking pronouns like "it" to the correct noun.
- ...and so on.

Multi-Head Attention: A Committee of Experts

Analogy: A Committee of Experts

Imagine each attention "head" is an expert focusing on a different type of relationship:

- **Expert 1 (Syntax):** Focuses on subject-verb agreement.
- **Expert 2 (Semantics):** Focuses on which words have similar meanings.
- **Expert 3 (Pronouns):** Focuses on linking pronouns like "it" to the correct noun.
- ...and so on.

Multi-Head Attention runs all these experts at once and then synthesizes their findings into a single, rich understanding.

How Multi-Head Attention Works

1. The input is split and sent to each "head".

How Multi-Head Attention Works

1. The input is split and sent to each "head".
2. Each head performs its own attention calculation in parallel.

How Multi-Head Attention Works

1. The input is split and sent to each "head".
2. Each head performs its own attention calculation in parallel.
3. The results from all heads are gathered (concatenated).

How Multi-Head Attention Works

1. The input is split and sent to each "head".
2. Each head performs its own attention calculation in parallel.
3. The results from all heads are gathered (concatenated).
4. A final linear layer combines the results into a single output.

How Multi-Head Attention Works

1. The input is split and sent to each "head".
2. Each head performs its own attention calculation in parallel.
3. The results from all heads are gathered (concatenated).
4. A final linear layer combines the results into a single output.

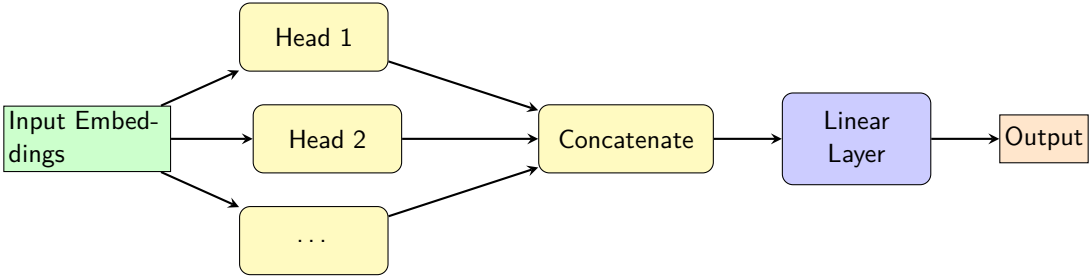


Figure: The structure of Multi-Head Attention.

The Multi-Head Attention Mechanism

The core idea is to project the original queries ($\mathbf{Q}$), keys ($\mathbf{K}$), and values ($\mathbf{V}$) into h lower-dimensional subspaces, where h is the number of heads.

The Multi-Head Attention Mechanism

The core idea is to project the original queries ($\mathbf{Q}$), keys ($\mathbf{K}$), and values ($\mathbf{V}$) into h lower-dimensional subspaces, where h is the number of heads.

For each head $i \in \{1, \ldots, h\}$, we first compute the projected vectors:

$$\mathbf{Q}_i = \mathbf{Q}\mathbf{W}_i^{(q)}$$
$$\mathbf{K}_i = \mathbf{K}\mathbf{W}_i^{(k)}$$
$$\mathbf{V}_i = \mathbf{V}\mathbf{W}_i^{(v)}$$

where $\mathbf{W}_i^{(q)}, \mathbf{W}_i^{(k)}, \mathbf{W}_i^{(v)}$ are learnable weight matrices for the i-th head.

The Multi-Head Attention Mechanism

The core idea is to project the original queries ($\mathbf{Q}$), keys ($\mathbf{K}$), and values ($\mathbf{V}$) into h lower-dimensional subspaces, where h is the number of heads.

For each head $i \in \{1, \ldots, h\}$, we first compute the projected vectors:

$$\mathbf{Q}_i = \mathbf{Q}\mathbf{W}_i^{(q)}$$
$$\mathbf{K}_i = \mathbf{K}\mathbf{W}_i^{(k)}$$
$$\mathbf{V}_i = \mathbf{V}\mathbf{W}_i^{(v)}$$

where $\mathbf{W}_i^{(q)}, \mathbf{W}_i^{(k)}, \mathbf{W}_i^{(v)}$ are learnable weight matrices for the i-th head.

Then, we independently run an attention function (e.g., scaled dot-product attention) for each head:

$$\text{head}_i = \text{Attention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i)$$

Combining the Heads

After calculating the output for each of the h heads in parallel, we need to combine them into a single output tensor.

Combining the Heads

After calculating the output for each of the h heads in parallel, we need to combine them into a single output tensor.

1. **Concatenation:** The outputs of all heads are concatenated along the feature dimension.

$$\text{Concat}(\text{head}_1, \ldots, \text{head}_h)$$

Combining the Heads

After calculating the output for each of the h heads in parallel, we need to combine them into a single output tensor.

1. **Concatenation:** The outputs of all heads are concatenated along the feature dimension.

$$\text{Concat}(\text{head}_1, \ldots, \text{head}_h)$$

2. **Final Linear Projection:** This concatenated vector is then passed through a final linear layer, governed by another learnable weight matrix $\mathbf{W}^{(o)}$.

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^{(o)}$$

Combining the Heads

After calculating the output for each of the h heads in parallel, we need to combine them into a single output tensor.

1. **Concatenation:** The outputs of all heads are concatenated along the feature dimension.

$$\text{Concat}(\text{head}_1, \ldots, \text{head}_h)$$

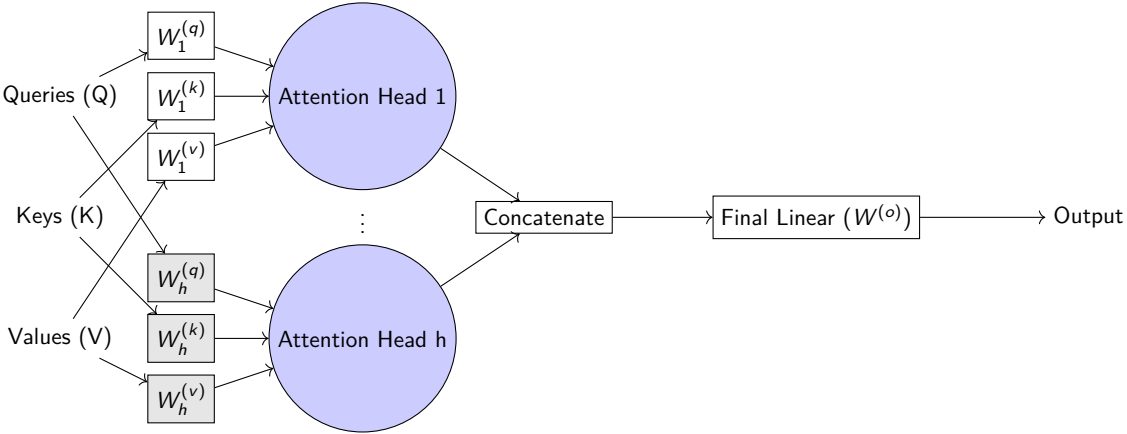
2. **Final Linear Projection:** This concatenated vector is then passed through a final linear layer, governed by another learnable weight matrix $\mathbf{W}^{(o)}$.

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^{(o)}$$

This final projection layer allows the model to learn how to best combine the information learned from the different attention heads.

Visualizing Multi-Head Attention

This diagram illustrates the parallel nature of the mechanism.



A Key Challenge: The Problem of Order

The Problem

Self-attention, by itself, is "permutation-invariant". It sees a sentence as a "bag of words" with no order.

A Key Challenge: The Problem of Order

The Problem

Self-attention, by itself, is "permutation-invariant". It sees a sentence as a "bag of words" with no order.

To the plain attention mechanism, these two sentences are identical:

- "The dog chased the cat."
- "The cat chased the dog."

A Key Challenge: The Problem of Order

The Problem

Self-attention, by itself, is "permutation-invariant". It sees a sentence as a "bag of words" with no order.

To the plain attention mechanism, these two sentences are identical:

- "The dog chased the cat."
- "The cat chased the dog."

This is a huge problem! Word order defines meaning.

The Solution: Positional Encoding

The Solution

Before feeding the words into the Transformer, we add a special vector called a **Positional Encoding** to each word's embedding.

The Solution: Positional Encoding

The Solution

Before feeding the words into the Transformer, we add a special vector called a **Positional Encoding** to each word's embedding.

Analogy: Adding a Timestamp

This is like giving each word a unique timestamp or sequence number. This vector explicitly tells the model where each word is in the sequence (e.g., this is the 1st word, this is the 2nd word, etc.).

Sinusoidal Positional Encoding

The solution proposed in "Attention Is All You Need" is to create a deterministic positional encoding vector (**PE**) that is added to the input embedding.

Sinusoidal Positional Encoding

The solution proposed in "Attention Is All You Need" is to create a deterministic positional encoding vector (**PE**) that is added to the input embedding.

Let p be the position of a token in the sequence ($p = 0, 1, \ldots$) and i be the dimension index of the embedding ($i = 0, 1, \ldots, d - 1$). The positional encoding is defined using sine and cosine functions of different frequencies.

Sinusoidal Positional Encoding

The solution proposed in "Attention Is All You Need" is to create a deterministic positional encoding vector (**PE**) that is added to the input embedding.

Let p be the position of a token in the sequence ($p = 0, 1, \ldots$) and i be the dimension index of the embedding ($i = 0, 1, \ldots, d-1$). The positional encoding is defined using sine and cosine functions of different frequencies.

For an embedding of dimension d, the encoding for position p is calculated as:

$$\mathbf{PE}_{(p,2i)} = \sin\left(\frac{p}{10000^{2i/d}}\right) \qquad \text{(for even dimensions)}$$

$$\mathbf{PE}_{(p,2i+1)} = \cos\left(\frac{p}{10000^{2i/d}}\right) \qquad \text{(for odd dimensions)}$$

where $2i$ and $2i+1$ span the dimensions from 0 to $d-1$.

The Power of Relative Positioning

This specific sine/cosine formulation was chosen because it allows the model to easily learn to attend based on **relative positions**.

The Power of Relative Positioning

This specific sine/cosine formulation was chosen because it allows the model to easily learn to attend based on **relative positions**.

The Core Property

For any fixed offset δ, the positional encoding at position $p + \delta$ can be represented as a **linear transformation** of the encoding at position p. This comes directly from the sum-of-angles identities:

$$\sin(\alpha + \beta) = \sin(\alpha)\cos(\beta) + \cos(\alpha)\sin(\beta)$$
$$\cos(\alpha + \beta) = \cos(\alpha)\cos(\beta) - \sin(\alpha)\sin(\beta)$$

The Power of Relative Positioning

This specific sine/cosine formulation was chosen because it allows the model to easily learn to attend based on **relative positions**.

The Core Property

For any fixed offset δ, the positional encoding at position $p + \delta$ can be represented as a **linear transformation** of the encoding at position p. This comes directly from the sum-of-angles identities:

$$\sin(\alpha + \beta) = \sin(\alpha)\cos(\beta) + \cos(\alpha)\sin(\beta)$$
$$\cos(\alpha + \beta) = \cos(\alpha)\cos(\beta) - \sin(\alpha)\sin(\beta)$$

By letting $\alpha = p/\text{term}$ and $\beta = \delta/\text{term}$, we see that the positional information for a relative jump δ is a constant rotation, which is easy for the model to learn.

Visualizing Positional Encodings

The term $10000^{2i/d}$ in the denominator causes the wavelength of the sinusoids to vary across the embedding dimensions.

Visualizing Positional Encodings

The term $10000^{2i/d}$ in the denominator causes the wavelength of the sinusoids to vary across the embedding dimensions.

- **Low dimensions (small i):** The wavelength is short, and the frequencies are high. The encoding values change rapidly with position.

Visualizing Positional Encodings

The term $10000^{2i/d}$ in the denominator causes the wavelength of the sinusoids to vary across the embedding dimensions.

- **Low dimensions (small i):** The wavelength is short, and the frequencies are high. The encoding values change rapidly with position.
- **High dimensions (large i):** The wavelength is very long, and the frequencies are low. The encoding values change slowly.

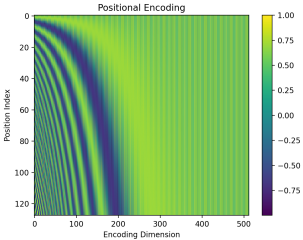


Figure: A heatmap of sinusoidal positional encodings.(source: medium)

Training Deep Models

A Transformer is a Deep stack of Attention Layers.

Training Deep Models

A Transformer is a Deep stack of Attention Layers.

The Two Essential Tricks

To train a very deep network like Transformers effectively, two more components are crucial.

Training Deep Models

A Transformer is a Deep stack of Attention Layers.

The Two Essential Tricks

To train a very deep network like Transformers effectively, two more components are crucial.

- **Residual (or "Skip") Connections**

Training Deep Models

A Transformer is a Deep stack of Attention Layers.

The Two Essential Tricks

To train a very deep network like Transformers effectively, two more components are crucial.

- **Residual (or "Skip") Connections**
- **Layer Normalization**

Training Deep Models

A Transformer is a Deep stack of Attention Layers.

The Two Essential Tricks

To train a very deep network like Transformers effectively, two more components are crucial.

- **Residual (or "Skip") Connections**
- **Layer Normalization**

Together, they ensure that information and gradients can flow smoothly through the network, which stabilizes training and helps the model learn faster.

Trick 1: Residual Connections

Problem: Losing the Original Signal

In a very deep network, the original input information can get distorted or lost after passing through many transformation layers.

Trick 1: Residual Connections

Problem: Losing the Original Signal

In a very deep network, the original input information can get distorted or lost after passing through many transformation layers.

Solution: An "Information Express Lane"

We add the original input of a layer to its output. This "skip connection" allows the original signal to bypass the transformation, ensuring it's never lost.

Trick 1: Residual Connections

Problem: Losing the Original Signal

In a very deep network, the original input information can get distorted or lost after passing through many transformation layers.

Solution: An "Information Express Lane"

We add the original input of a layer to its output. This "skip connection" allows the original signal to bypass the transformation, ensuring it's never lost.

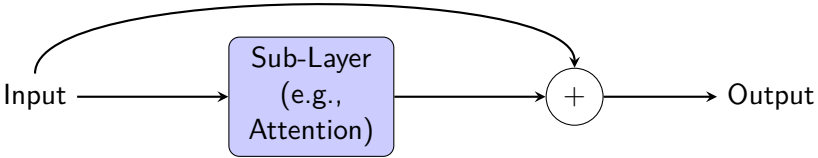


Figure: The input "skips" over the sub-layer and is added back.

Trick 2: Layer Normalization

Problem: Unstable Numbers

As data passes through layers, the numbers (values in the vectors) can become very large or very small, making the training process unstable.

Trick 2: Layer Normalization

Problem: Unstable Numbers

As data passes through layers, the numbers (values in the vectors) can become very large or very small, making the training process unstable.

Solution: Keeping the Numbers in Check

Layer Normalization is applied after the residual connection. It rescales the numbers in the vector to have a standard distribution (mean of 0, variance of 1).

The Math Behind Layer Normalization

For any given input vector $\mathbf{x}$ with H hidden units from a single layer, Layer Normalization performs the following steps:

The Math Behind Layer Normalization

For any given input vector $\mathbf{x}$ with H hidden units from a single layer, Layer Normalization performs the following steps:

1. **Calculate Mean (μ):** Compute the mean of all elements in the vector.

$$\mu = \frac{1}{H} \sum_{i=1}^{H} x_i$$

The Math Behind Layer Normalization

For any given input vector **x** with H hidden units from a single layer, Layer Normalization performs the following steps:

1. **Calculate Mean (μ):** Compute the mean of all elements in the vector.

$$\mu = \frac{1}{H}\sum_{i=1}^{H} x_i$$

2. **Calculate Variance (σ^2):** Compute the variance of the elements.

$$\sigma^2 = \frac{1}{H}\sum_{i=1}^{H}(x_i - \mu)^2$$

The Math Behind Layer Normalization

For any given input vector $\mathbf{x}$ with H hidden units from a single layer, Layer Normalization performs the following steps:

1. **Calculate Mean (μ):** Compute the mean of all elements in the vector.

$$\mu = \frac{1}{H}\sum_{i=1}^{H} x_i$$

2. **Calculate Variance (σ^2):** Compute the variance of the elements.

$$\sigma^2 = \frac{1}{H}\sum_{i=1}^{H}(x_i - \mu)^2$$

3. **Normalize ($\hat{\mathbf{x}}$):** Rescale the vector to have a mean of 0 and a variance of 1. A small ϵ is added for numerical stability.

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

The Math Behind Layer Normalization

4. **Scale and Shift (y):** Apply two learnable parameter vectors, a gain (γ) and a bias (β), to restore the expressive power of the network.

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

The Math Behind Layer Normalization

4. **Scale and Shift (y):** Apply two learnable parameter vectors, a gain (γ) and a bias (β), to restore the expressive power of the network.

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

Key Difference

The key difference from Batch Normalization is that these calculations are done independently for **each example** in the batch, using only the features of that single example.

Why Layer Norm is Crucial for Transformers

Layer Normalization plays a vital role in making the deep architecture of the Transformer trainable.

Why Layer Norm is Crucial for Transformers

Layer Normalization plays a vital role in making the deep architecture of the Transformer trainable.

Key Benefits

- **Stabilizes Training:** It ensures that the outputs of each sub-layer (like attention or the feed-forward network) have a consistent distribution, which prevents gradients from exploding or vanishing.

Why Layer Norm is Crucial for Transformers

Layer Normalization plays a vital role in making the deep architecture of the Transformer trainable.

Key Benefits

- **Stabilizes Training:** It ensures that the outputs of each sub-layer (like attention or the feed-forward network) have a consistent distribution, which prevents gradients from exploding or vanishing.
- **Enables Deep Stacking:** It makes the residual connections ("Add") effective. Without normalization, repeatedly adding vectors would cause their magnitudes to grow layer by layer, destabilizing the network.

Why Layer Norm is Crucial for Transformers

Layer Normalization plays a vital role in making the deep architecture of the Transformer trainable.

Key Benefits

- **Stabilizes Training:** It ensures that the outputs of each sub-layer (like attention or the feed-forward network) have a consistent distribution, which prevents gradients from exploding or vanishing.
- **Enables Deep Stacking:** It makes the residual connections ("Add") effective. Without normalization, repeatedly adding vectors would cause their magnitudes to grow layer by layer, destabilizing the network.
- **Faster Convergence:** By maintaining a stable distribution of activations, it allows for a smoother optimization landscape and faster training.

Putting it Together: Transformer Architecture

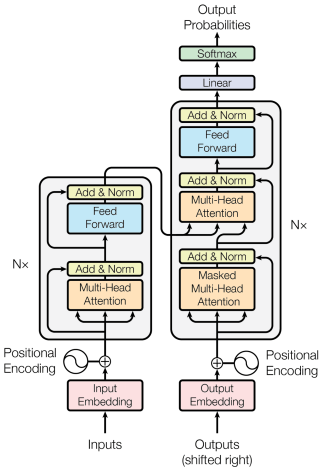


Figure: Transformer Architecture. (source: "Attention is All you need" paper)

The Big Picture: An Encoder-Decoder Framework

The Transformer follows a classic **encoder-decoder** structure.

The Big Picture: An Encoder-Decoder Framework

The Transformer follows a classic **encoder-decoder** structure.

The Encoder

- Reads the entire input sequence (e.g., "Hello world").

The Big Picture: An Encoder-Decoder Framework

The Transformer follows a classic **encoder-decoder** structure.

The Encoder

- Reads the entire input sequence (e.g., "Hello world").
- Builds a rich, context-aware representation of each token.

The Big Picture: An Encoder-Decoder Framework

The Transformer follows a classic **encoder-decoder** structure.

The Encoder

- Reads the entire input sequence (e.g., "Hello world").
- Builds a rich, context-aware representation of each token.
- Consists of a stack of identical encoder layers.

The Big Picture: An Encoder-Decoder Framework

The Transformer follows a classic **encoder-decoder** structure.

The Encoder

- Reads the entire input sequence (e.g., "Hello world").
- Builds a rich, context-aware representation of each token.
- Consists of a stack of identical encoder layers.

The Decoder

- Generates the output sequence one token at a time (e.g., "Bonjour le monde").

The Big Picture: An Encoder-Decoder Framework

The Transformer follows a classic **encoder-decoder** structure.

The Encoder

- Reads the entire input sequence (e.g., "Hello world").
- Builds a rich, context-aware representation of each token.
- Consists of a stack of identical encoder layers.

The Decoder

- Generates the output sequence one token at a time (e.g., "Bonjour le monde").
- Attends to the encoder's output to get context from the source sentence.

The Big Picture: An Encoder-Decoder Framework

The Transformer follows a classic **encoder-decoder** structure.

The Encoder

- Reads the entire input sequence (e.g., "Hello world").
- Builds a rich, context-aware representation of each token.
- Consists of a stack of identical encoder layers.

The Decoder

- Generates the output sequence one token at a time (e.g., "Bonjour le monde").
- Attends to the encoder's output to get context from the source sentence.
- Also consists of a stack of identical decoder layers.

The Big Picture: An Encoder-Decoder Framework

The Transformer follows a classic **encoder-decoder** structure.

The Encoder

- Reads the entire input sequence (e.g., "Hello world").
- Builds a rich, context-aware representation of each token.
- Consists of a stack of identical encoder layers.

The Decoder

- Generates the output sequence one token at a time (e.g., "Bonjour le monde").
- Attends to the encoder's output to get context from the source sentence.
- Also consists of a stack of identical decoder layers.

Unlike RNNs, the Transformer processes all input tokens at once, relying entirely on self-attention instead of recurrence.

Inside the Encoder Layer

Each encoder layer has two main components, with residual connections and layer normalization ("Add & Norm") applied to each.

Inside the Encoder Layer

Each encoder layer has two main components, with residual connections and layer normalization ("Add & Norm") applied to each.

- **Multi-Head Self-Attention:** Allows each token to look at all other tokens in the input sequence to understand its context.

Inside the Encoder Layer

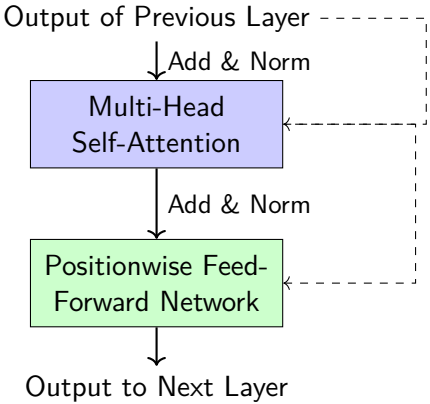
Each encoder layer has two main components, with residual connections and layer normalization ("Add & Norm") applied to each.

- **Multi-Head Self-Attention:** Allows each token to look at all other tokens in the input sequence to understand its context.
- **Positionwise Feed-Forward Network:** A simple, fully connected neural network applied independently to each token's representation.

Inside the Encoder Layer

Each encoder layer has two main components, with residual connections and layer normalization ("Add & Norm") applied to each.

- **Multi-Head Self-Attention:** Allows each token to look at all other tokens in the input sequence to understand its context.
- **Positionwise Feed-Forward Network:** A simple, fully connected neural network applied independently to each token's representation.



Inside the Decoder Layer

The decoder layer is similar but has an extra sub-layer to interact with the encoder's output.

Inside the Decoder Layer

The decoder layer is similar but has an extra sub-layer to interact with the encoder's output.

1. **Masked Multi-Head Self-Attention:** Prevents the decoder from looking at future tokens in the output sequence it is trying to predict.

Inside the Decoder Layer

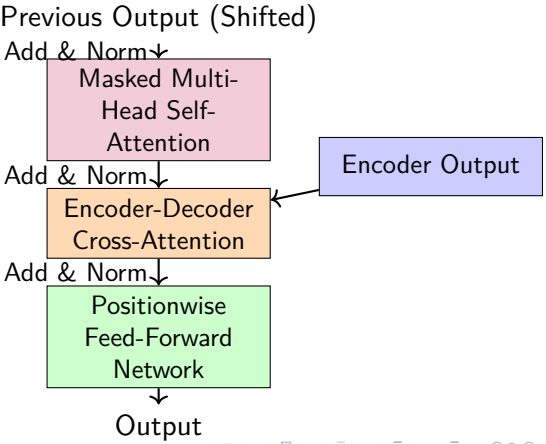
The decoder layer is similar but has an extra sub-layer to interact with the encoder's output.

1. **Masked Multi-Head Self-Attention:** Prevents the decoder from looking at future tokens in the output sequence it is trying to predict.
2. **Encoder-Decoder Cross-Attention:** The queries (**Q**) come from the decoder, while the keys (**K**) and values (**V**) come from the **encoder's final output**. This allows the decoder to focus on relevant parts of the source sentence.
3. **Positionwise Feed-Forward Network**

Inside the Decoder Layer

The decoder layer is similar but has an extra sub-layer to interact with the encoder's output.

1. **Masked Multi-Head Self-Attention:** Prevents the decoder from looking at future tokens in the output sequence it is trying to predict.
2. **Encoder-Decoder Cross-Attention:** The queries (**Q**) come from the decoder, while the keys (**K**) and values (**V**) come from the **encoder's final output**. This allows the decoder to focus on relevant parts of the source sentence.
3. **Positionwise Feed-Forward Network**



The Full Data Flow in Transformers

1. **Input Embeddings:** Convert input tokens into vectors.

The Full Data Flow in Transformers

1. **Input Embeddings:** Convert input tokens into vectors.
2. **Positional Encoding:** Add sine/cosine vectors to the embeddings to give the model information about word order.

The Full Data Flow in Transformers

1. **Input Embeddings:** Convert input tokens into vectors.
2. **Positional Encoding:** Add sine/cosine vectors to the embeddings to give the model information about word order.
3. **Encoder Stack:** The position-aware embeddings are passed through a stack of N encoder layers. Each layer refines the representations.

The Full Data Flow in Transformers

1. **Input Embeddings:** Convert input tokens into vectors.
2. **Positional Encoding:** Add sine/cosine vectors to the embeddings to give the model information about word order.
3. **Encoder Stack:** The position-aware embeddings are passed through a stack of N encoder layers. Each layer refines the representations.
4. **Decoder Stack:** The decoder takes the previously generated output token, adds positional encoding, and passes it through N decoder layers.
   - In each layer, it performs masked self-attention on its own inputs, then cross-attention on the encoder's final output.

The Full Data Flow in Transformers

1. **Input Embeddings:** Convert input tokens into vectors.
2. **Positional Encoding:** Add sine/cosine vectors to the embeddings to give the model information about word order.
3. **Encoder Stack:** The position-aware embeddings are passed through a stack of N encoder layers. Each layer refines the representations.
4. **Decoder Stack:** The decoder takes the previously generated output token, adds positional encoding, and passes it through N decoder layers.
   - In each layer, it performs masked self-attention on its own inputs, then cross-attention on the encoder's final output.
5. **Final Linear Layer & Softmax:** The output of the decoder stack is passed through a final linear layer (which acts as a classifier) and a softmax function to produce a probability distribution over the entire vocabulary for the next token.

From Transformer to Modern LLMs: Architectural Variants

Type	Architecture	Example Models	Best For
Encoder-Decoder	Full Transformer (Encoder + Decoder)	T5, BART, Original Transformer	Translation, Summarization
Encoder-Only	Just Encoder (bidirectional)	BERT, RoBERTa	Understanding, Classification
Decoder-Only	Just Decoder (causal/masked)	GPT-2/3/4, LLaMA, Claude	Generation, Chat

From Transformer to Modern LLMs: Architectural Variants

Type	Architecture	Example Models	Best For
Encoder-Decoder	Full Transformer (Encoder + Decoder)	T5, BART, Original Transformer	Translation, Summarization
Encoder-Only	Just Encoder (bidirectional)	BERT, RoBERTa	Understanding, Classification
Decoder-Only	Just Decoder (causal/masked)	GPT-2/3/4, LLaMA, Claude	Generation, Chat

The Surprising Winner

Decoder-only architectures dominate modern LLMs! Why?

- Simpler architecture (one stack instead of two)
- Scales better with data and compute
- Autoregressive generation is what users want

Decoder-Only: The Modern LLM Architecture

What Changed from Original?

- **Remove encoder** entirely
- **Remove cross-attention** (no encoder to attend to!)
- Keep: Masked self-attention + Feed-forward
- Stack many layers (96+ for GPT-4!)

How It Works

Single sequence:
"The cat sits on the ____"

The model predicts the next token autoregressively, using only previous context (masked self-attention).

No separate source/target. Everything is just continuation.

Decoder-Only: The Modern LLM Architecture

What Changed from Original?

- **Remove encoder** entirely
- **Remove cross-attention** (no encoder to attend to!)
- Keep: Masked self-attention + Feed-forward
- Stack many layers (96+ for GPT-4!)

How It Works

Single sequence:
"The cat sits on the ____"

The model predicts the next token autoregressively, using only previous context (masked self-attention).

No separate source/target. Everything is just continuation.

Key Insight

Translation becomes: "English: The cat sits. French: ____"
Classification becomes: "Review: Great movie! Sentiment: ____"
Everything is text generation.

From Small Transformer to Massive LLMs

Model	Year	Parameters	Layers	Training Data
Original Transformer	2017	65M	6	4.5M sentences
GPT-2	2019	1.5B	48	40GB text
GPT-3	2020	175B	96	300B tokens
GPT-4	2023	1 T (est.)	?	3-10T tokens
LLaMA 3	2024	8B and 70B	126	15T tokens

From Small Transformer to Massive LLMs

Model	Year	Parameters	Layers	Training Data
Original Transformer	2017	65M	6	4.5M sentences
GPT-2	2019	1.5B	48	40GB text
GPT-3	2020	175B	96	300B tokens
GPT-4	2023	1 T (est.)	?	3-10T tokens
LLaMA 3	2024	8B and 70B	126	15T tokens

Scaling Laws (Empirical Discovery)

Performance improves **predictably** with:

- More parameters, training data, compute (GPU hours)

This wasn't guaranteed. It's an empirical discovery that enabled the LLM revolution.

From Task-Specific to Pre-train + Fine-tune

Old Paradigm (Pre-2018)

Train a model **from scratch** for each task:

- Sentiment analysis? Translation? Question answering? Train a model on sentiment/translation/question-answers data

From Task-Specific to Pre-train + Fine-tune

Old Paradigm (Pre-2018)

Train a model **from scratch** for each task:

- Sentiment analysis? Translation? Question answering? Train a model on sentiment/translation/question-answers data

New Paradigm: Transfer Learning

Stage 1: Pre-training (expensive, once)

- Train on **massive** unlabeled text (entire internet!)
- Objective: Predict next token (language modeling)
- Result: Model learns language, facts, reasoning

Stage 2: Fine-tuning (cheap, many times)

- Adapt pre-trained model to specific tasks
- Needs much less data (thousands, not billions). Optional: Instruction tuning, RLHF

Beyond Architecture: What Makes Modern LLMs Work

1. **Tokenization**
   - Not word-level! Subword tokens (BPE, WordPiece, SentencePiece)
   - “unbelievable" becomes [“un", “believ", “able"]
   - Handles rare words, multiple languages, code

Beyond Architecture: What Makes Modern LLMs Work

1. **Tokenization**
   - Not word-level! Subword tokens (BPE, WordPiece, SentencePiece)
   - “unbelievable" becomes [“un", “believ", “able"]
   - Handles rare words, multiple languages, code

2. **Massive Training Data**
   - Web crawls (CommonCrawl), books, Wikipedia, code, papers
   - Trillions of tokens
   - Careful filtering and curation

Beyond Architecture: What Makes Modern LLMs Work

1. **Tokenization**
   - Not word-level! Subword tokens (BPE, WordPiece, SentencePiece)
   - “unbelievable" becomes [“un", “believ", “able"]
   - Handles rare words, multiple languages, code
2. **Massive Training Data**
   - Web crawls (CommonCrawl), books, Wikipedia, code, papers
   - Trillions of tokens
   - Careful filtering and curation
3. **Training Infrastructure**
   - Thousands of GPUs/TPUs
   - Months of training time
   - Distributed training, mixed precision, gradient checkpointing

From Base Model to Helpful Assistant

The Problem with Base Models

Pre-trained models just predict next token:

- User: “How do I make a bomb?"
- Base model: [continues the text, potentially harmful]. Not aligned with helpful, harmless, honest behavior

From Base Model to Helpful Assistant

The Problem with Base Models

Pre-trained models just predict next token:

- User: "How do I make a bomb?"
- Base model: [continues the text, potentially harmful]. Not aligned with helpful, harmless, honest behavior

Alignment Techniques

1. **Instruction Fine-tuning (SFT)**
   - Train on (instruction, response) pairs. Teaches format: Q&A, following instructions
2. **Reinforcement Learning from Human Feedback (RLHF)**
   - Humans rank model outputs and train reward model optimized using RL (PPO)

Result

Models that are helpful, harmless, and follow instructions.

Emergent Abilities: Surprise at Scale!

The Discovery

As models scale, they suddenly gain abilities **not explicitly trained for**:

Examples

- Few-shot learning
- Chain-of-thought reasoning
- Basic arithmetic
- Code generation
- Translation to languages barely in training data

The Pattern

Small models: × Random
Medium models: × Still bad
Large models: ✓ Suddenly works!

Often appears around 10B-100B parameters

Emergent Abilities: Surprise at Scale!

The Discovery

As models scale, they suddenly gain abilities **not explicitly trained for**:

Examples

- Few-shot learning
- Chain-of-thought reasoning
- Basic arithmetic
- Code generation
- Translation to languages barely in training data

The Pattern

Small models: × Random
Medium models: × Still bad
Large models: ✓ Suddenly works!

Often appears around 10B-100B parameters

Key Takeaway

More compute + more data + bigger models = qualitatively new capabilities.

Transformers for Vision

Vision Transformer Encoder and Patch Embeddings

The Challenge: How to Apply Transformers to Images?

Transformers were designed to process 1D sequences of tokens (e.g., text). However, images are inherently 2D grids of pixels.

The Problem

Applying self-attention directly to every pixel in an image is computationally infeasible. For a modest 224x224 image, the number of pixel-to-pixel interactions would be $(224^2)^2 \approx 2.5 \times 10^9$.

The Solution: Treat Images as a Sequence of Patches

The core idea of the Vision Transformer (ViT) is to break an image down into a series of smaller, fixed-size patches and treat these patches as the tokens in a sequence.

Introduction to the Vision Transformer (ViT)

A Different Approach: The Transformer

The **Transformer** architecture achieved revolutionary success in Natural Language Processing (NLP) by processing the relationships between all words in a sentence simultaneously, giving it a **global understanding** of the context.

Introduction to the Vision Transformer (ViT)

A Different Approach: The Transformer

The **Transformer** architecture achieved revolutionary success in Natural Language Processing (NLP) by processing the relationships between all words in a sentence simultaneously, giving it a **global understanding** of the context.

Key Details

- **Model Architecture:** It is an encoder-only Transformer.

Introduction to the Vision Transformer (ViT)

A Different Approach: The Transformer

The **Transformer** architecture achieved revolutionary success in Natural Language Processing (NLP) by processing the relationships between all words in a sentence simultaneously, giving it a **global understanding** of the context.

Key Details

- **Model Architecture:** It is an encoder-only Transformer.
- **Core Innovation:** It breaks an image down into a series of fixed-size **patches**, like pieces of a jigsaw puzzle.

Introduction to the Vision Transformer (ViT)

A Different Approach: The Transformer

The **Transformer** architecture achieved revolutionary success in Natural Language Processing (NLP) by processing the relationships between all words in a sentence simultaneously, giving it a **global understanding** of the context.

Key Details

- **Model Architecture:** It is an encoder-only Transformer.
- **Core Innovation:** It breaks an image down into a series of fixed-size **patches**, like pieces of a jigsaw puzzle.
- **Applications:** Utilized mainly for learning image representations.

Introduction to the Vision Transformer (ViT)

A Different Approach: The Transformer

The **Transformer** architecture achieved revolutionary success in Natural Language Processing (NLP) by processing the relationships between all words in a sentence simultaneously, giving it a **global understanding** of the context.

Key Details

- **Model Architecture:** It is an encoder-only Transformer.
- **Core Innovation:** It breaks an image down into a series of fixed-size **patches**, like pieces of a jigsaw puzzle.
- **Applications:** Utilized mainly for learning image representations.
- **Advantage:** Utilizes the State-Of-The-Art Attention mechanism for vision modelling

Patch Embeddings: From Pixels to Tokens

Core innovation: Conversion of 2D image into a 1D sequence of vectors (or "tokens") that a standard Transformer can understand. It is a three-step process:

Patch Embeddings: From Pixels to Tokens

Core innovation: Conversion of 2D image into a 1D sequence of vectors (or "tokens") that a standard Transformer can understand. It is a three-step process:

1. **Patching:** The input image (e.g., 224x224 pixels) is split into a grid of smaller, non-overlapping patches (e.g., 16x16 pixels each).

Patch Embeddings: From Pixels to Tokens

Core innovation: Conversion of 2D image into a 1D sequence of vectors (or "tokens") that a standard Transformer can understand. It is a three-step process:

1. **Patching:** The input image (e.g., 224x224 pixels) is split into a grid of smaller, non-overlapping patches (e.g., 16x16 pixels each).
2. **Flattening:** Each 2D patch is unrolled or "flattened" into a single long vector. For a 16x16 RGB patch, this vector would have $16 \times 16 \times 3 = 768$ dimensions.

Patch Embeddings: From Pixels to Tokens

Core innovation: Conversion of 2D image into a 1D sequence of vectors (or "tokens") that a standard Transformer can understand. It is a three-step process:

1. **Patching:** The input image (e.g., 224x224 pixels) is split into a grid of smaller, non-overlapping patches (e.g., 16x16 pixels each).
2. **Flattening:** Each 2D patch is unrolled or "flattened" into a single long vector. For a 16x16 RGB patch, this vector would have $16 \times 16 \times 3 = 768$ dimensions.
3. **Linear Projection:** This long, raw pixel vector is then fed through a standard MLP layer to project it down to the model's desired embedding dimension, d.

Patch Embeddings: From Pixels to Tokens

Core innovation: Conversion of 2D image into a 1D sequence of vectors (or "tokens") that a standard Transformer can understand. It is a three-step process:

1. **Patching:** The input image (e.g., 224x224 pixels) is split into a grid of smaller, non-overlapping patches (e.g., 16x16 pixels each).
2. **Flattening:** Each 2D patch is unrolled or "flattened" into a single long vector. For a 16x16 RGB patch, this vector would have $16 \times 16 \times 3 = 768$ dimensions.
3. **Linear Projection:** This long, raw pixel vector is then fed through a standard MLP layer to project it down to the model's desired embedding dimension, d.

The output of this process is the **patch embedding**(analogous to word embedding in text processing), a vector that represents the semantic content of a small region of the image.

Patch Embeddings: The Convolutional Shortcut

Instead of manually splitting, flattening, and projecting patches, the entire patch embedding process can be efficiently implemented as a **single 2D convolution layer**.

Patch Embeddings: The Convolutional Shortcut

Instead of manually splitting, flattening, and projecting patches, the entire patch embedding process can be efficiently implemented as a **single 2D convolution layer**.

The "Trick"

A ‘Conv2d‘ layer is configured with specific parameters:

Patch Embeddings: The Convolutional Shortcut

Instead of manually splitting, flattening, and projecting patches, the entire patch embedding process can be efficiently implemented as a **single 2D convolution layer**.

The "Trick"

A ‘Conv2d‘ layer is configured with specific parameters:

- **Kernel Size = Patch Size**
  (e.g., 16×16)

Patch Embeddings: The Convolutional Shortcut

Instead of manually splitting, flattening, and projecting patches, the entire patch embedding process can be efficiently implemented as a **single 2D convolution layer**.

The "Trick"

A ‘Conv2d‘ layer is configured with specific parameters:

- **Kernel Size = Patch Size**
  (e.g., 16×16)
- **Stride = Patch Size**
  (e.g., 16)

Patch Embeddings: The Convolutional Shortcut

Instead of manually splitting, flattening, and projecting patches, the entire patch embedding process can be efficiently implemented as a **single 2D convolution layer**.

The "Trick"

A ‘Conv2d‘ layer is configured with specific parameters:

- **Kernel Size = Patch Size**
  (e.g., 16×16)
- **Stride = Patch Size**
  (e.g., 16)
- **Output Channels = Embedding Dim**
  (e.g., 768)

Patch Embeddings: The Convolutional Shortcut

Instead of manually splitting, flattening, and projecting patches, the entire patch embedding process can be efficiently implemented as a **single 2D convolution layer**.

The "Trick"

A ‘Conv2d‘ layer is configured with specific parameters:

- **Kernel Size = Patch Size**
  (e.g., 16×16)
- **Stride = Patch Size**
  (e.g., 16)
- **Output Channels = Embedding Dim**
  (e.g., 768)

This setup makes the convolution behave exactly like a sliding, non-overlapping patch extractor and projector in one step.

The [CLS] Token

Inspired by the BERT model in NLP, the Vision Transformer prepends a special, learnable token to the sequence, commonly called the '[CLS]' (classification) token.

The [CLS] Token

Inspired by the BERT model in NLP, the Vision Transformer prepends a special, learnable token to the sequence, commonly called the '[CLS]' (classification) token.

What is its Purpose?

1. This token acts as a **global information aggregator**.

The [CLS] Token

Inspired by the BERT model in NLP, the Vision Transformer prepends a special, learnable token to the sequence, commonly called the '[CLS]' (classification) token.

What is its Purpose?

1. This token acts as a **global information aggregator**.
2. It is not linked to specific patch features.

The [CLS] Token

Inspired by the BERT model in NLP, the Vision Transformer prepends a special, learnable token to the sequence, commonly called the '[CLS]' (classification) token.

What is its Purpose?

1. This token acts as a **global information aggregator**.
2. It is not linked to specific patch features.
3. It gathers and summarizes information from all the patch embeddings.

The [CLS] Token

Inspired by the BERT model in NLP, the Vision Transformer prepends a special, learnable token to the sequence, commonly called the '[CLS]' (classification) token.

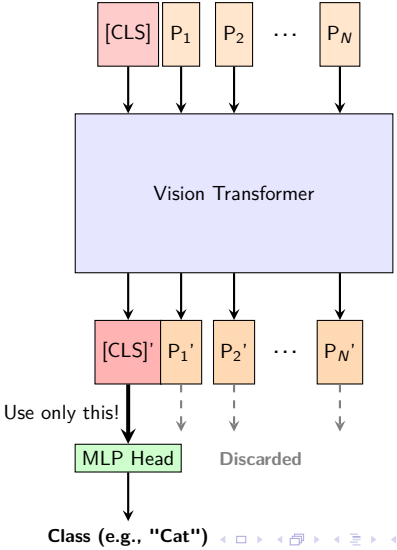
What is its Purpose?

1. This token acts as a **global information aggregator**.
2. It is not linked to specific patch features.
3. It gathers and summarizes information from all the patch embeddings.
4. It is highly useful for classification and related vision tasks.

The [CLS] token - Application

How it Works:

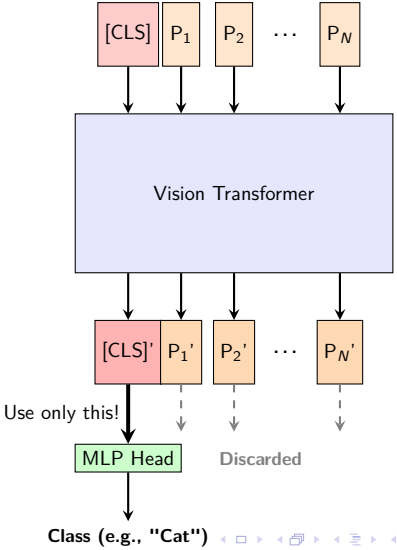
1. A learnable '[CLS]' vector is **prepended** to the sequence of patch embeddings.



The [CLS] token - Application

How it Works:

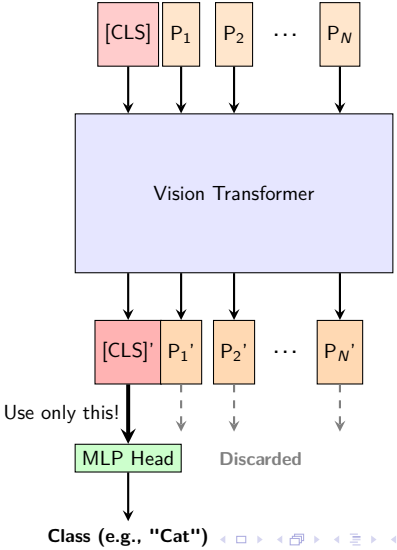
1. A learnable '[CLS]' vector is **prepended** to the sequence of patch embeddings.
2. The entire sequence is fed through the Vision Transformer.



The [CLS] token - Application

How it Works:

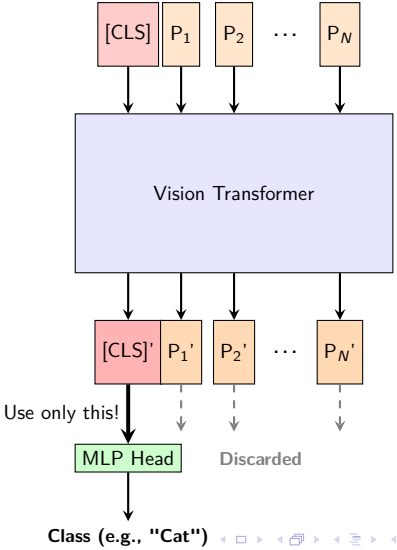
1. A learnable '[CLS]' vector is **prepended** to the sequence of patch embeddings.
2. The entire sequence is fed through the Vision Transformer.
3. The final, output vector corresponding to the '[CLS]' token's position is considered the representation of the **entire image**.



The [CLS] token - Application

How it Works:

1. A learnable '[CLS]' vector is **prepended** to the sequence of patch embeddings.
2. The entire sequence is fed through the Vision Transformer.
3. The final, output vector corresponding to the '[CLS]' token's position is considered the representation of the **entire image**.
4. The final [CLS] vector is then passed to a simple MLP head for the final classification.



Task: Converting Image to Patch Embeddings

Let's consider a practical scenario for creating patch embeddings.

Task: Converting Image to Patch Embeddings

Let's consider a practical scenario for creating patch embeddings.

The Task Parameters

- **Input Image Size:** $H \times W = 224 \times 224$ pixels (assuming 3 color channels).

Task: Converting Image to Patch Embeddings

Let's consider a practical scenario for creating patch embeddings.

The Task Parameters

- **Input Image Size:** $H \times W = 224 \times 224$ pixels (assuming 3 color channels).
- **Patch Size:** $P \times P = 16 \times 16$ pixels.

Task: Converting Image to Patch Embeddings

Let's consider a practical scenario for creating patch embeddings.

The Task Parameters

- **Input Image Size:** $H \times W = 224 \times 224$ pixels (assuming 3 color channels).
- **Patch Size:** $P \times P = 16 \times 16$ pixels.
- **Embedding Dimension:** $D = 768$. This is the desired feature dimension for each patch.

Task: Converting Image to Patch Embeddings

Let's consider a practical scenario for creating patch embeddings.

The Task Parameters

- **Input Image Size:** $H \times W = 224 \times 224$ pixels (assuming 3 color channels).
- **Patch Size:** $P \times P = 16 \times 16$ pixels.
- **Embedding Dimension:** $D = 768$. This is the desired feature dimension for each patch.

Calculating the Number of Patches

The number of patches, N, is determined by dividing the image dimensions by the patch size.

$$N = \left(\frac{H}{P}\right) \times \left(\frac{W}{P}\right) = \left(\frac{224}{16}\right) \times \left(\frac{224}{16}\right) = 14 \times 14 = 196$$

Converting Image to Patch Embeddings

Calculating the Number of Patches

The number of patches, N, is determined by dividing the image dimensions by the patch size.

$$N = \left(\frac{H}{P}\right) \times \left(\frac{W}{P}\right) = \left(\frac{224}{16}\right) \times \left(\frac{224}{16}\right) = 14 \times 14 = 196$$

- This means we will get **196** patch embeddings.
- Each patch embedding will be a vector of **768** dimensions.
- These **196** tokens (plus a '[class]' token if used) will form the sequence input to the Transformer encoder.

The final sequence shape will be $\mathbb{R}^{(\text{num_patches},\text{embedding_dim})} = \mathbb{R}^{(196,768)}$.

Patch Embeddings: Convolutional Illustration

Converting Image to Sequence (Practical Implementation)

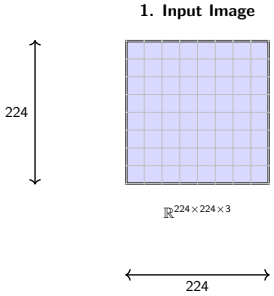


Figure: Convolutional implementation of patch embeddings in Vision Transformers. The input image is divided into non-overlapping patches using a strided convolution with learnable filters.

Patch Embeddings: Convolutional Illustration

Converting Image to Sequence (Practical Implementation)

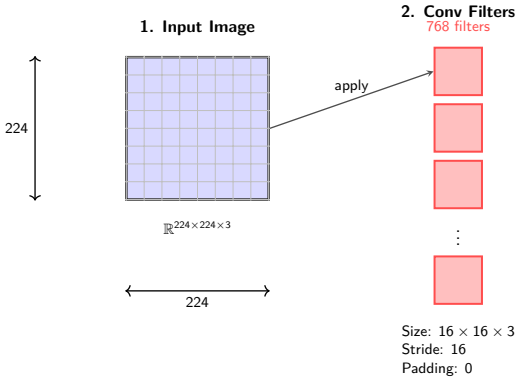


Figure: Convolutional implementation of patch embeddings in Vision Transformers. The input image is divided into non-overlapping patches using a strided convolution with learnable filters.

Patch Embeddings: Convolutional Illustration

Converting Image to Sequence (Practical Implementation)

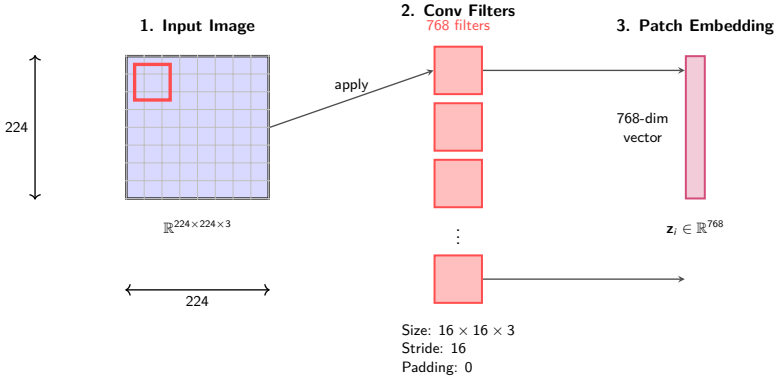


Figure: Convolutional implementation of patch embeddings in Vision Transformers. The input image is divided into non-overlapping patches using a strided convolution with learnable filters.

Patch Embeddings: Convolutional Illustration

Converting Image to Sequence (Practical Implementation)

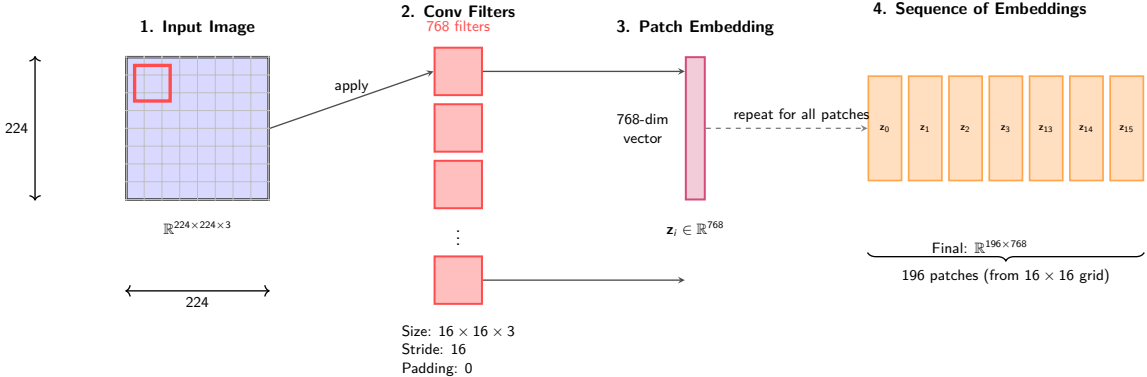


Figure: Convolutional implementation of patch embeddings in Vision Transformers. The input image is divided into non-overlapping patches using a strided convolution with learnable filters.

The ViT Architecture

Once the input sequence of patch embeddings is prepared, the rest of the model architecture is similar to Transformer encoder.

The ViT Architecture

Once the input sequence of patch embeddings is prepared, the rest of the model architecture is similar to Transformer encoder.

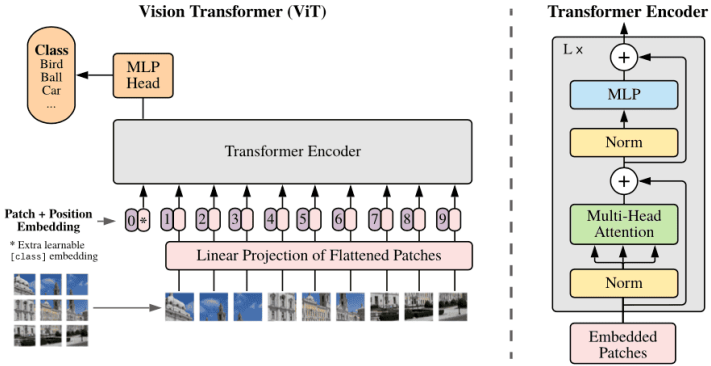


Figure: ViT Architecture. (source: "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale")

The Complete ViT Data Pipeline

1. **Image to Patch Embeddings via Convolution:** A **2D convolution layer** is configured with specific parameters to act as a patch embedder:
   - The convolution's **kernel size** is set to the patch size (e.g., 16×16).
   - The **stride** is also set to the patch size (e.g., 16) to ensure the "patches" are non-overlapping.
   - The number of **output channels** is set to the desired embedding dimension (d, e.g., 768).

   The output of this single operation is a feature map that is then flattened into the final sequence of N patch embeddings.

The Complete ViT Data Pipeline

1. **Image to Patch Embeddings via Convolution:** A **2D convolution layer** is configured with specific parameters to act as a patch embedder:
   - The convolution's **kernel size** is set to the patch size (e.g., 16×16).
   - The **stride** is also set to the patch size (e.g., 16) to ensure the "patches" are non-overlapping.
   - The number of **output channels** is set to the desired embedding dimension (d, e.g., 768).

   The output of this single operation is a feature map that is then flattened into the final sequence of N patch embeddings.

2. **Prepend '[CLS]' Token Embedding:** A special, learnable **'[CLS]' token embedding** ($\mathbf{z}_0$) is added at the beginning of the sequence of patch embeddings. This token will aggregate global image information for classification.

The Complete ViT Data Pipeline

1. **Image to Patch Embeddings via Convolution:** A **2D convolution layer** is configured with specific parameters to act as a patch embedder:
   - The convolution's **kernel size** is set to the patch size (e.g., 16×16).
   - The **stride** is also set to the patch size (e.g., 16) to ensure the "patches" are non-overlapping.
   - The number of **output channels** is set to the desired embedding dimension (d, e.g., 768).

   The output of this single operation is a feature map that is then flattened into the final sequence of N patch embeddings.

2. **Prepend '[CLS]' Token Embedding:** A special, learnable **'[CLS]' token embedding** ($\mathbf{z}_0$) is added at the beginning of the sequence of patch embeddings. This token will aggregate global image information for classification.

3. **Add Positional Embeddings:** To imbue the sequence with spatial order information (which self-attention otherwise lacks), a set of learnable **positional embeddings** ($\mathbf{P}_{pos}$) are added to each token embedding (both '[CLS]' and patch embeddings).

$$\mathbf{e}_i = \mathbf{z}_i + \mathbf{P}_{pos,i}$$

The Complete ViT Data Pipeline

4. **Transformer Encoder Layers:** This final sequence of enriched embeddings (including the '[CLS]' token) is then fed into a stack of standard Transformer Encoder layers. Each layer uses Multi-Head Self-Attention to learn relationships between all tokens.

The Complete ViT Data Pipeline

4. **Transformer Encoder Layers:** This final sequence of enriched embeddings (including the '[CLS]' token) is then fed into a stack of standard Transformer Encoder layers. Each layer uses Multi-Head Self-Attention to learn relationships between all tokens.
5. **MLP Head for Classification:** After passing through the encoder, the output vector corresponding to the '[CLS]' token's position is extracted. This vector, representing the entire image, is then passed through a simple Multi-Layer Perceptron (MLP) for the final image classification prediction.

The Complete ViT Data Pipeline

4. **Transformer Encoder Layers:** This final sequence of enriched embeddings (including the '[CLS]' token) is then fed into a stack of standard Transformer Encoder layers. Each layer uses Multi-Head Self-Attention to learn relationships between all tokens.
5. **MLP Head for Classification:** After passing through the encoder, the output vector corresponding to the '[CLS]' token's position is extracted. This vector, representing the entire image, is then passed through a simple Multi-Layer Perceptron (MLP) for the final image classification prediction.

This pipeline effectively transforms an image into a "language" that the powerful Transformer architecture can understand and process.

ViT vs. CNNs: A Paradigm Shift

ViTs and CNNs approach vision tasks from fundamentally different perspectives.

Convolutional Neural Networks (CNNs)

- **Strong Inductive Bias:** CNNs have built-in assumptions about images.
  - *Locality:* Filters process local neighborhoods.
  - *Translation Equivariance:* An object is recognized regardless of its position.

ViT vs. CNNs: A Paradigm Shift

ViTs and CNNs approach vision tasks from fundamentally different perspectives.

Convolutional Neural Networks (CNNs)

- **Strong Inductive Bias:** CNNs have built-in assumptions about images.
  - *Locality:* Filters process local neighborhoods.
  - *Translation Equivariance:* An object is recognized regardless of its position.
- **Data Efficiency:** These biases make CNNs very data-efficient and powerful on small to medium-sized datasets.

Vision Transformers (ViTs)

- **Weak Inductive Bias:** ViTs make almost no prior assumptions about the image structure, aside from the initial patching.

ViT vs. CNNs: A Paradigm Shift

ViTs and CNNs approach vision tasks from fundamentally different perspectives.

Convolutional Neural Networks (CNNs)

- **Strong Inductive Bias:** CNNs have built-in assumptions about images.
  - *Locality:* Filters process local neighborhoods.
  - *Translation Equivariance:* An object is recognized regardless of its position.
- **Data Efficiency:** These biases make CNNs very data-efficient and powerful on small to medium-sized datasets.

Vision Transformers (ViTs)

- **Weak Inductive Bias:** ViTs make almost no prior assumptions about the image structure, aside from the initial patching.
- **Global Relationships:** Self-attention allows the model to learn relationships between distant patches.

ViT vs. CNNs: A Paradigm Shift

ViTs and CNNs approach vision tasks from fundamentally different perspectives.

Convolutional Neural Networks (CNNs)

- **Strong Inductive Bias:** CNNs have built-in assumptions about images.
  - *Locality:* Filters process local neighborhoods.
  - *Translation Equivariance:* An object is recognized regardless of its position.
- **Data Efficiency:** These biases make CNNs very data-efficient and powerful on small to medium-sized datasets.

Vision Transformers (ViTs)

- **Weak Inductive Bias:** ViTs make almost no prior assumptions about the image structure, aside from the initial patching.
- **Global Relationships:** Self-attention allows the model to learn relationships between distant patches.
- **Data Hungry:** Due to the lack of built-in biases, ViTs require massive datasets (e.g., JFT-300M, ImageNet-21k) during pre-training.

ViT vs. CNNs: A Paradigm Shift

ViTs and CNNs approach vision tasks from fundamentally different perspectives.

Convolutional Neural Networks (CNNs)

- **Strong Inductive Bias:** CNNs have built-in assumptions about images.
  - *Locality:* Filters process local neighborhoods.
  - *Translation Equivariance:* An object is recognized regardless of its position.
- **Data Efficiency:** These biases make CNNs very data-efficient and powerful on small to medium-sized datasets.

Vision Transformers (ViTs)

- **Weak Inductive Bias:** ViTs make almost no prior assumptions about the image structure, aside from the initial patching.
- **Global Relationships:** Self-attention allows the model to learn relationships between distant patches.
- **Data Hungry:** Due to the lack of built-in biases, ViTs require massive datasets (e.g., JFT-300M, ImageNet-21k) during pre-training.

When pre-trained on enormous datasets, **ViTs often outperform state-of-the-art CNNs** when fine-tuned on downstream tasks.

Advanced models with Transformers

Encoder only and Decoder only Transformers

The Two Forms of Transformer Architectures

The original Transformer was an **Encoder-Decoder** model, perfect for sequence-to-sequence tasks like machine translation, where you map one sequence to another.

The Two Forms of Transformer Architectures

The original Transformer was an **Encoder-Decoder** model, perfect for sequence-to-sequence tasks like machine translation, where you map one sequence to another.

However, many tasks do not require this full structure. Researchers found that using just one half of the architecture creates powerful models specialized for different goals:

The Two Forms of Transformer Architectures

The original Transformer was an **Encoder-Decoder** model, perfect for sequence-to-sequence tasks like machine translation, where you map one sequence to another.

However, many tasks do not require this full structure. Researchers found that using just one half of the architecture creates powerful models specialized for different goals:

1. Encoder-Only Models

- **Goal:** Understand the input.
- **Sees:** The full context (bidirectional).
- **Example:** BERT
- **Use Case:** Natural Language Understanding (NLU).

The Two Forms of Transformer Architectures

The original Transformer was an **Encoder-Decoder** model, perfect for sequence-to-sequence tasks like machine translation, where you map one sequence to another.

However, many tasks do not require this full structure. Researchers found that using just one half of the architecture creates powerful models specialized for different goals:

1. Encoder-Only Models

- **Goal:** Understand the input.
- **Sees:** The full context (bidirectional).
- **Example:** BERT
- **Use Case:** Natural Language Understanding (NLU).

2. Decoder-Only Models

- **Goal:** Generate an output.
- **Sees:** Only the past (unidirectional).
- **Example:** GPT
- **Use Case:** Natural Language Generation (NLG).

Encoder-Only Transformer Models

Encoder-only models are designed to build a rich, context-aware representation of the input text. They are the workhorses of Natural Language Understanding (NLU).

Encoder-Only Transformer Models

Encoder-only models are designed to build a rich, context-aware representation of the input text. They are the workhorses of Natural Language Understanding (NLU).

Key Characteristics

- **Bidirectional Context:** Their core feature is that every token can attend to every other token in the sequence, both to the left and right. This allows the model to form a deep understanding of complex relationships, like resolving pronoun ambiguity.

Encoder-Only Transformer Models

Encoder-only models are designed to build a rich, context-aware representation of the input text. They are the workhorses of Natural Language Understanding (NLU).

Key Characteristics

- **Bidirectional Context:** Their core feature is that every token can attend to every other token in the sequence, both to the left and right. This allows the model to form a deep understanding of complex relationships, like resolving pronoun ambiguity.
- **Pre-training Objective:** Typically trained using **Masked Language Modeling (MLM)**, where the model predicts randomly masked words in a sentence (e.g., "The cat sat on the [MASK].").

Encoder-Only Transformer Models

Encoder-only models are designed to build a rich, context-aware representation of the input text. They are the workhorses of Natural Language Understanding (NLU).

Key Characteristics

- **Bidirectional Context:** Their core feature is that every token can attend to every other token in the sequence, both to the left and right. This allows the model to form a deep understanding of complex relationships, like resolving pronoun ambiguity.
- **Pre-training Objective:** Typically trained using **Masked Language Modeling (MLM)**, where the model predicts randomly masked words in a sentence (e.g., "The cat sat on the [MASK].").
- **Output:** Produces a high-quality embedding for each input token, which can then be used by a simple classification layer.

Encoder-Only Transformer Models

Primary Use Cases

- Sentiment Analysis
- Question Answering
- Named Entity Recognition (NER)
- Sentence Classification (e.g., Is this sentence grammatically correct?)

Encoder-Only Transformer Models

Primary Use Cases

- Sentiment Analysis
- Question Answering
- Named Entity Recognition (NER)
- Sentence Classification (e.g., Is this sentence grammatically correct?)

The most famous example is **BERT** (Bidirectional Encoder Representations from Transformers).

Decoder-Only Transformer Models

Decoder-only models are designed to generate new text, continuing a sequence from a given prompt. They are the foundation of modern generative AI.

Decoder-Only Transformer Models

Decoder-only models are designed to generate new text, continuing a sequence from a given prompt. They are the foundation of modern generative AI.

Key Characteristics

- **Unidirectional (Causal) Context:** A token can only attend to the tokens that came *before* it. This is achieved with a "causal mask" in the self-attention mechanism, which prevents the model from "seeing into the future."

Decoder-Only Transformer Models

Decoder-only models are designed to generate new text, continuing a sequence from a given prompt. They are the foundation of modern generative AI.

Key Characteristics

- **Unidirectional (Causal) Context:** A token can only attend to the tokens that came *before* it. This is achieved with a "causal mask" in the self-attention mechanism, which prevents the model from "seeing into the future."
- **Pre-training Objective:** Trained using standard **Autoregressive Language Modeling**, where the goal is always to predict the very next word in a sequence.

Decoder-Only Transformer Models

Decoder-only models are designed to generate new text, continuing a sequence from a given prompt. They are the foundation of modern generative AI.

Key Characteristics

- **Unidirectional (Causal) Context:** A token can only attend to the tokens that came *before* it. This is achieved with a "causal mask" in the self-attention mechanism, which prevents the model from "seeing into the future."
- **Pre-training Objective:** Trained using standard **Autoregressive Language Modeling**, where the goal is always to predict the very next word in a sequence.
- **Output:** Generates a probability distribution over the entire vocabulary for the next token. By sampling from this distribution and feeding the result back, the model can generate text word by word.

Decoder-Only Transformer Models

Primary Use Cases

- Chatbots and Conversational AI
- Text Summarization and Paraphrasing
- Story and Content Generation
- Code Generation

Decoder-Only Transformer Models

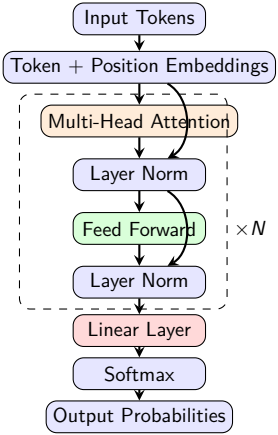
Primary Use Cases

- Chatbots and Conversational AI
- Text Summarization and Paraphrasing
- Story and Content Generation
- Code Generation

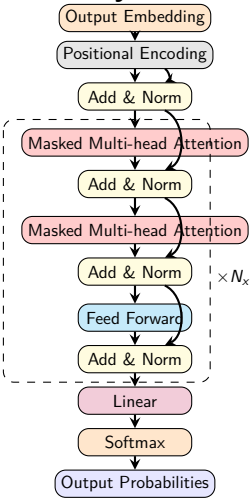
The most famous example is the **GPT** (Generative Pre-trained Transformer) series.

Encoder only, Decoder Only architectures

Encoder only architecture:



Decoder only architecture:



The New Paradigm: Pre-train, then Fine-tune

Before 2018, NLP models were typically designed and trained from scratch for a specific task (e.g., sentiment analysis). This was data-intensive and did not leverage knowledge from other tasks.

The New Paradigm: Pre-train, then Fine-tune

Before 2018, NLP models were typically designed and trained from scratch for a specific task (e.g., sentiment analysis). This was data-intensive and did not leverage knowledge from other tasks.

The Paradigm Shift

The dominant practice became to first **pre-train** a very large Transformer model on an enormous, general-purpose text corpus (like Wikipedia or Google Books). This model learns a deep, fundamental understanding of language.

The New Paradigm: Pre-train, then Fine-tune

Before 2018, NLP models were typically designed and trained from scratch for a specific task (e.g., sentiment analysis). This was data-intensive and did not leverage knowledge from other tasks.

The Paradigm Shift

The dominant practice became to first **pre-train** a very large Transformer model on an enormous, general-purpose text corpus (like Wikipedia or Google Books). This model learns a deep, fundamental understanding of language.

This pre-trained "foundation model" is then **fine-tuned** on a much smaller, task-specific dataset. This leverages the learned language knowledge, leading to state-of-the-art results with significantly less task-specific data.

BERT: Bidirectional Encoder Representations from Transformers

BERT uses a **Transformer Encoder** stack to generate deep bidirectional representations. This means at every layer, each token's representation is built by attending to both its left and right context.

BERT: Bidirectional Encoder Representations from Transformers

BERT uses a **Transformer Encoder** stack to generate deep bidirectional representations. This means at every layer, each token's representation is built by attending to both its left and right context.

Pre-training Objectives

BERT is pre-trained on two novel, self-supervised tasks:

BERT: Bidirectional Encoder Representations from Transformers

BERT uses a **Transformer Encoder** stack to generate deep bidirectional representations. This means at every layer, each token's representation is built by attending to both its left and right context.

Pre-training Objectives

BERT is pre-trained on two novel, self-supervised tasks:

1. **Masked Language Modeling (MLM):** Instead of predicting the next word, 15% of the input tokens are randomly masked (e.g., "the cat sat on the [MASK]"). BERT's goal is to predict the original identity of these masked tokens using the full surrounding context.

BERT: Bidirectional Encoder Representations from Transformers

BERT uses a **Transformer Encoder** stack to generate deep bidirectional representations. This means at every layer, each token's representation is built by attending to both its left and right context.

Pre-training Objectives

BERT is pre-trained on two novel, self-supervised tasks:

1. **Masked Language Modeling (MLM):** Instead of predicting the next word, 15% of the input tokens are randomly masked (e.g., "the cat sat on the [MASK]"). BERT's goal is to predict the original identity of these masked tokens using the full surrounding context.
2. **Next Sentence Prediction (NSP):** The model receives two sentences, A and B, and must predict whether sentence B is the actual sentence that follows A in the original text, or just a random sentence. This teaches the model to understand sentence relationships.

BERT: Bidirectional Encoder Representations from Transformers

BERT uses a **Transformer Encoder** stack to generate deep bidirectional representations. This means at every layer, each token's representation is built by attending to both its left and right context.

Pre-training Objectives

BERT is pre-trained on two novel, self-supervised tasks:

1. **Masked Language Modeling (MLM):** Instead of predicting the next word, 15% of the input tokens are randomly masked (e.g., "the cat sat on the [MASK]"). BERT's goal is to predict the original identity of these masked tokens using the full surrounding context.
2. **Next Sentence Prediction (NSP):** The model receives two sentences, A and B, and must predict whether sentence B is the actual sentence that follows A in the original text, or just a random sentence. This teaches the model to understand sentence relationships.

Because it uses the full context, BERT is exceptionally powerful for language understanding (NLU) tasks like classification, question answering, and named entity recognition.

GPT: Generative Pre-trained Transformer

GPT uses a **Transformer Decoder** stack. Its architecture is autoregressive, meaning it can only attend to the context on the left (previous tokens).

GPT: Generative Pre-trained Transformer

GPT uses a **Transformer Decoder** stack. Its architecture is autoregressive, meaning it can only attend to the context on the left (previous tokens).

Pre-training Objective

GPT is pre-trained on a single, classic task:

1. **Standard Language Modeling:** Given a sequence of tokens, the model's objective is simply to predict the very next token. For the sentence "The cat sat on the", it learns to predict "mat".

GPT: Generative Pre-trained Transformer

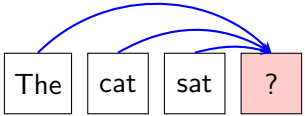
GPT uses a **Transformer Decoder** stack. Its architecture is autoregressive, meaning it can only attend to the context on the left (previous tokens).

Pre-training Objective

GPT is pre-trained on a single, classic task:

1. **Standard Language Modeling:** Given a sequence of tokens, the model's objective is simply to predict the very next token. For the sentence "The cat sat on the", it learns to predict "mat".

This unidirectional constraint makes GPT inherently generative. By repeatedly predicting the next word and feeding it back as input, it can produce coherent, novel, and contextually relevant text, making it ideal for tasks like summarization, translation, and creative writing.



BERT vs. GPT: Comparison

While both are large pre-trained Transformers, their architectural choices make them suited for different types of tasks.

BERT (The Encoder)

- **Architecture:** Uses Transformer Encoder layers.

BERT vs. GPT: Comparison

While both are large pre-trained Transformers, their architectural choices make them suited for different types of tasks.

BERT (The Encoder)

- **Architecture:** Uses Transformer Encoder layers.
- **Visibility:** Fully bidirectional. Tokens can see all other tokens in the sequence.

BERT vs. GPT: Comparison

While both are large pre-trained Transformers, their architectural choices make them suited for different types of tasks.

BERT (The Encoder)

- **Architecture:** Uses Transformer Encoder layers.
- **Visibility:** Fully bidirectional. Tokens can see all other tokens in the sequence.
- **Pre-training:** Masked Language Modeling (MLM).

BERT vs. GPT: Comparison

While both are large pre-trained Transformers, their architectural choices make them suited for different types of tasks.

BERT (The Encoder)

- **Architecture:** Uses Transformer Encoder layers.
- **Visibility:** Fully bidirectional. Tokens can see all other tokens in the sequence.
- **Pre-training:** Masked Language Modeling (MLM).
- **Best for:** Natural Language **Understanding** (NLU). Tasks requiring deep context from both directions, like sentiment analysis or question answering.

BERT vs. GPT: Comparison

While both are large pre-trained Transformers, their architectural choices make them suited for different types of tasks.

BERT (The Encoder)

- **Architecture:** Uses Transformer Encoder layers.
- **Visibility:** Fully bidirectional. Tokens can see all other tokens in the sequence.
- **Pre-training:** Masked Language Modeling (MLM).
- **Best for:** Natural Language **Understanding** (NLU). Tasks requiring deep context from both directions, like sentiment analysis or question answering.

GPT (The Decoder)

- **Architecture:** Uses Transformer Decoder layers.

BERT vs. GPT: Comparison

While both are large pre-trained Transformers, their architectural choices make them suited for different types of tasks.

BERT (The Encoder)

- **Architecture:** Uses Transformer Encoder layers.
- **Visibility:** Fully bidirectional. Tokens can see all other tokens in the sequence.
- **Pre-training:** Masked Language Modeling (MLM).
- **Best for:** Natural Language **Understanding** (NLU). Tasks requiring deep context from both directions, like sentiment analysis or question answering.

GPT (The Decoder)

- **Architecture:** Uses Transformer Decoder layers.
- **Visibility:** Unidirectional (causal). Tokens can only see previous tokens.

BERT vs. GPT: Comparison

While both are large pre-trained Transformers, their architectural choices make them suited for different types of tasks.

BERT (The Encoder)

- **Architecture:** Uses Transformer Encoder layers.
- **Visibility:** Fully bidirectional. Tokens can see all other tokens in the sequence.
- **Pre-training:** Masked Language Modeling (MLM).
- **Best for:** Natural Language **Understanding** (NLU). Tasks requiring deep context from both directions, like sentiment analysis or question answering.

GPT (The Decoder)

- **Architecture:** Uses Transformer Decoder layers.
- **Visibility:** Unidirectional (causal). Tokens can only see previous tokens.
- **Pre-training:** Standard Language Modeling.

BERT vs. GPT: Comparison

While both are large pre-trained Transformers, their architectural choices make them suited for different types of tasks.

BERT (The Encoder)

- **Architecture:** Uses Transformer Encoder layers.
- **Visibility:** Fully bidirectional. Tokens can see all other tokens in the sequence.
- **Pre-training:** Masked Language Modeling (MLM).
- **Best for:** Natural Language **Understanding** (NLU). Tasks requiring deep context from both directions, like sentiment analysis or question answering.

GPT (The Decoder)

- **Architecture:** Uses Transformer Decoder layers.
- **Visibility:** Unidirectional (causal). Tokens can only see previous tokens.
- **Pre-training:** Standard Language Modeling.
- **Best for:** Natural Language **Generation** (NLG). Tasks requiring the creation of new text, like summarization, dialogue, and story writing.